

C# / .NET — Tập 3: Nâng cao

Web API · EF Core · hiệu năng · Docker

2026-09-28

Mục lục

Tập 3 — C# Nâng cao: backend thật

Chương 1 — ASP.NET Core Web API (Minimal API)

Chương 2 — EF Core với SQLite

Chương 3 — Hiệu năng: Span, bộ nhớ, BenchmarkDotNet

Chương 4 — Concurrency (xử lý đồng thời)

Chương 5 — Clean architecture

Chương 6 — Logging và observability

Chương 7 — Docker và dự án cuối

GLOSSARY — Thuật ngữ (Tiếng Anh → Tiếng Việt)

Tập 3 — C# Nâng cao: backend thật

Tập này dựng backend thật: Web API, database, hiệu năng, concurrency, kiến trúc, logging và Docker. Tập kết thúc bằng một dự án Web API hoàn chỉnh.

Mục lục

#	Chương	Ý chính	Ví dụ chạy được
1	ASP.NET Core Web API	Minimal API, routing, DTO, validation, status code, middleware	<code>V3Ch01.MinimalApi</code>
2	EF Core với SQLite	<code>DbContext</code> , entity, query, migration/ <code>EnsureCreated</code> , tracking	<code>V3Ch02.EfCore</code>
3	Hiệu năng: Span, bộ nhớ, BenchmarkDotNet	<code>Span<T></code> , <code>stackalloc</code> , <code>ArrayPool</code> , GC, benchmark	<code>V3Ch03.Spans</code> , <code>V3Ch03.Bench</code>
4	Concurrency	<code>Task.Run</code> , <code>Parallel</code> , <code>lock</code> / <code>Lock</code> , <code>Interlocked</code> , <code>Channel<T></code>	<code>V3Ch04.Concurrency</code>
5	Clean architecture	Domain / Application / Infrastructure / Api, dependency rule	xem <code>final-project/</code>
6	Logging và observability	<code>ILogger</code> , structured logging, health checks, <code>OpenTelemetry</code>	<code>V3Ch06.Logging</code>
7	Docker và dự án cuối	Dockerfile multi-stage, chạy container, tổng kết dự án	<code>final-project/</code>

Dự án cuối: `final-project/` — Web API quản lý công việc (TaskBoard) với EF Core SQLite, xUnit (unit + integration), Dockerfile.

Chạy ví dụ

```
cd books/csharp
./vol3-advanced/run-examples.sh
```

Quy ước trình bày

- Thuật ngữ tiếng Anh giữ nguyên, giải thích tiếng Việt ở lần đầu; xem thêm Glossary cuối sách.
- Code và output trong sách được chép tự động từ file thật trong `examples/` sau mỗi lần chạy.
- Kiểm tra hiển thị tiếng Việt: ă ằ ẳ ẵ ơ ư ơ ư Đ (nếu các chữ này hiện đúng, font đã đủ dấu).

Chương 1 — ASP.NET Core Web API (Minimal API)

Mục tiêu

- Hiểu cấu trúc một ứng dụng ASP.NET Core: builder → services → app → middleware → endpoints.
- Viết endpoint với Minimal API: `MapGet`, `MapPost`, `MapDelete`, `MapGroup`, route constraint.
- Nhận dữ liệu (route, query, body JSON) và trả kết quả đúng HTTP status với `TypedResults`.
- Dùng validation có sẵn của .NET 10 (`AddValidation`) và *Problem Details* cho lỗi.
- Viết middleware đơn giản.

Giải thích đơn giản

ASP.NET Core là framework web của .NET. Có hai cách viết API:

- **Minimal API**: khai báo endpoint bằng lambda — giống Express/Fastify trong Node.js.
- **Controller** (MVC): class kế thừa `ControllerBase`, attribute `[HttpGet]` — giống Spring `@RestController` hoặc NestJS.

Bộ sách dùng Minimal API vì ngắn, nhanh và là hướng Microsoft đang đầu tư mạnh.

Express (Node)	Spring Boot	ASP.NET Core Minimal API
<code>const app = express()</code>	<code>@SpringBootApplication</code>	<code>var builder = WebApplication.CreateBuilder(args)</code>
<code>app.use(mw)</code>	<code>Filter / HandlerInterceptor</code>	<code>app.Use(...)</code> (middleware)
<code>app.get('/x/:id', h)</code>	<code>@GetMapping("/x/{id}")</code>	<code>app.MapGet("/x/{id:int}", h)</code>
<code>express.Router()</code>	<code>@RequestMapping</code> trên class	<code>app.MapGroup("/api/x")</code>
<code>res.status(201).json(x)</code>	<code>ResponseEntity.created(...)</code>	<code>TypedResults.Created(url, x)</code>
<code>class-validator / zod</code>	Bean Validation <code>@Valid</code>	<code>DataAnnotations + AddValidation()</code> (.NET 10)

Ví dụ

Ví dụ này **tự chạy server**, **tự gọi API bằng `HttpClient`** rồi tắt — nên output ở dưới là request/response thật.

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <!-- Web SDK: includes ASP.NET Core (Kestrel, routing, DI, logging, config). -->

</Project>

using System.Collections.Concurrent;
using System.ComponentModel.DataAnnotations;
using System.Diagnostics;
using System.Net.Http.Json;
using Microsoft.AspNetCore.Http.HttpResults;
```

```

var builder = WebApplication.CreateBuilder(args);
builder.Logging.SetMinimumLevel(LogLevel.Warning); // giữ output ngắn gọn
builder.WebHost.UseUrls("http://127.0.0.1:5199");

builder.Services.AddSingleton<ProductStore>();
builder.Services.AddValidation(); // .NET 10: validate DataAnnotations trong Minimal API
builder.Services.AddProblemDetails();

var app = builder.Build();

// Middleware tự viết: đo thời gian xử lý và thêm header
app.Use(async (context, next) =>
{
    var sw = Stopwatch.StartNew();
    context.Response.OnStarting(() =>
    {
        context.Response.Headers["X-Elapsed-Ms"] = sw.ElapsedMilliseconds.ToString();
        return Task.CompletedTask;
    });
    await next(context);
});

// Endpoints
app.MapGet("/", () => "Xin chào từ ASP.NET Core!");

var products = app.MapGroup("/api/products");

products.MapGet("/", (ProductStore store, string? search, int page = 1, int pageSize = 10) =>
{
    var query = store.All();
    if (!string.IsNullOrEmpty(search))
        query = query.Where(p => p.Name.Contains(search, StringComparison.OrdinalIgnoreCase));
    return TypedResults.Ok(query.Skip((page - 1) * pageSize).Take(pageSize).ToList());
});

products.MapGet("/{id:int}", Results<Ok<ProductDto>, NotFound> (int id, ProductStore store) =>
    store.Find(id) is { } p ? TypedResults.Ok(p) : TypedResults.NotFound());

products.MapPost("/", (CreateProductRequest request, ProductStore store) =>
{
    var created = store.Add(request);
    return TypedResults.Created($"/api/products/{created.Id}", created);
});

products.MapDelete("/{id:int}", Results<NoContent, NotFound> (int id, ProductStore store) =>
    store.Remove(id) ? TypedResults.NoContent() : TypedResults.NotFound());

await app.StartAsync();

// ---- Client: gọi chính API vừa chạy ----
using var http = new HttpClient { BaseAddress = new Uri("http://127.0.0.1:5199") };

async Task Show(string label, Func<Task<HttpResponseMessage>> call)
{
    var response = await call();
    var body = await response.Content.ReadAsStringAsync();
    Console.WriteLine($"{label} -> {(int)response.StatusCode} {response.StatusCode}");
    if (body.Length > 0) Console.WriteLine($"    {body}");
}

await Show("GET /", () => http.GetAsync("/"));
await Show("POST /api/products", () => http.PostAsJsonAsync("/api/products", new { name = "Bàn  
phím cơ", price = 1_200_000 }));
await Show("POST /api/products", () => http.PostAsJsonAsync("/api/products", new { name =  
"Chuột", price = 250_000 }));
await Show("POST (dữ liệu sai)", () => http.PostAsJsonAsync("/api/products", new { name = "",  
price = -5 }));

```

```

        await Show("GET /api/products?search=bàn", () => http.GetAsync("/api/products?search=b%C3%A0n"));
        await Show("GET /api/products/2", () => http.GetAsync("/api/products/2"));
        await Show("GET /api/products/99", () => http.GetAsync("/api/products/99"));
        await Show("GET /api/products/abc", () => http.GetAsync("/api/products/abc"));
        await Show("DELETE /api/products/1", () => http.DeleteAsync("/api/products/1"));

var head = await http.GetAsync("/api/products");
Console.WriteLine($"Header X-Elapsed-Ms có mặt? {head.Headers.Contains("X-Elapsed-Ms")}");

await app.StopAsync();

// ---- Models ----
public record CreateProductRequest(
    [property: Required, StringLength(100, MinimumLength = 1)] string Name,
    [property: Range(0, 1_000_000_000)] decimal Price);

public record ProductDto(int Id, string Name, decimal Price);

public class ProductStore
{
    private readonly ConcurrentDictionary<int, ProductDto> _items = new();
    private int _nextId;

    public IEnumerable<ProductDto> All() => _items.Values.OrderBy(p => p.Id);
    public ProductDto? Find(int id) => _items.GetValueOrDefault(id);
    public bool Remove(int id) => _items.TryRemove(id, out _);

    public ProductDto Add(CreateProductRequest request)
    {
        var product = new ProductDto(Interlocked.Increment(ref _nextId), request.Name, request.Price);
        _items[product.Id] = product;
        return product;
    }
}

```

Output thật:

```

GET / -> 200 OK
Xin chào từ ASP.NET Core!
POST /api/products -> 201 Created
{"id":1,"name":"Bàn phím cơ","price":1200000}
POST /api/products -> 201 Created
{"id":2,"name":"Chuột","price":250000}
POST (dữ liệu sai) -> 400 BadRequest
{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.1","title":"One or more validation errors occurred.","status":400,"errors":{"Name":["The Name field is required."],"Price":["The field Price must be between 0 and 1000000000."]},"traceId":"00-789ece916d56ef39916aa0f6a4dd0ffa-c2794034ee276aad-00"}
GET /api/products?search=bàn -> 200 OK
[{"id":1,"name":"Bàn phím cơ","price":1200000}]
GET /api/products/2 -> 200 OK
{"id":2,"name":"Chuột","price":250000}
GET /api/products/99 -> 404 NotFound
GET /api/products/abc -> 404 NotFound
DELETE /api/products/1 -> 204 NoContent
Header X-Elapsed-Ms có mặt? True

```

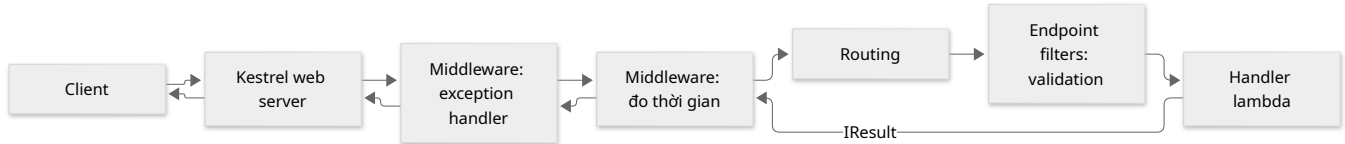
Để ý:

- POST dữ liệu sai → **400** kèm `errors` cho từng trường, do `AddValidation()` tự kiểm tra `[Required]`, `[Range]` — không cần viết `if` nào.
- GET `/api/products/abc` → **404** vì route constraint `{id:int}` không khớp.

- `DELETE` thành công → **204 No Content** (không có body).

Đi sâu

Vòng đời một request



- **Middleware** chạy theo thứ tự đăng ký, "bọc" nhau như củ hành: code trước `await next()` chạy khi request đi vào, code sau chạy khi response đi ra.
- **Endpoint filter** chỉ chạy cho endpoint cụ thể (validation, kiểm tra API key...).

Parameter binding — tham số đến từ đâu?

Nguồn	Ví dụ	Quy tắc
Route	<code>/ {id:int} → int id</code>	trùng tên với route
Query	<code>?search=bàn&page=2 → string? search, int page = 1</code>	kiểu đơn giản, không có trong route
Body (JSON)	<code>CreateProductRequest request</code>	kiểu phức tạp (class/record)
DI	<code>ProductStore store</code>	kiểu đã đăng ký trong <code>builder.Services</code>
Đặc biệt	<code>HttpContext, Cancellation token</code>	framework tự truyền

Có thể ghi rõ bằng `[FromRoute]`, `[FromQuery]`, `[FromBody]`, `[FromServices]`, `[FromHeader]`.

`TypedResults` và `Results<...>`

`TypedResults.Ok(x)` trả về kiểu cụ thể `Ok<T>`, giúp OpenAPI biết chính xác response. Khi một endpoint có nhiều kết quả, khai báo kiểu trả về `Results<Ok<ProductDto>, NotFound>` — compiler kiểm tra bạn chỉ trả về đúng các loại đã khai báo.

Status code nên dùng

Tình huống	Status
Đọc thành công	200 OK
Tạo mới	201 Created + header <code>Location</code>
Xóa / cập nhật không trả body	204 No Content
Dữ liệu gửi lên sai	400 Bad Request (Problem Details)
Chưa đăng nhập / sai key	401 Unauthorized
Không tìm thấy	404 Not Found
Xung đột trạng thái nghiệp vụ	409 Conflict

Tình huống	Status
Lỗi server	500 Internal Server Error

Problem Details (RFC 7807 / RFC 9457)

Định dạng JSON chuẩn cho lỗi HTTP: `type`, `title`, `status`, `detail`, `errors` ... Tài liệu ASP.NET Core gọi đây là chuẩn RFC 7807; RFC 9457 là bản thay thế mới hơn (**UNVERIFIED** — không mở được `rfc-editor.org` từ môi trường viết sách). `builder.Services.AddProblemDetails()` bật định dạng này cho toàn ứng dụng. Frontend (Angular) chỉ cần một hàm xử lý lỗi cho mọi API.

Lỗi và bẫy thường gặp

- **Thứ tự middleware sai:** `ExceptionHandler` phải đứng đầu để bắt lỗi của mọi middleware sau.
- **Trả entity EF Core trực tiếp:** lộ cột nội bộ, vòng lặp JSON (navigation 2 chiều). Luôn trả **DTO**.
- **Quên `CancellationToken`** trong endpoint gọi DB → request bị hủy nhưng query vẫn chạy.
- **Dùng `async void`** trong handler → lỗi không bắt được. Luôn trả `Task/Task<T>`.
- **Validation chỉ ở frontend:** API phải tự kiểm tra; client có thể là bất kỳ ai.
- **AddValidation và nhiều assembly:** source generator chỉ quét assembly gọi `AddValidation()` (theo release notes ASP.NET Core 10).

Tóm tắt

- `WebApplication.CreateBuilder` → đăng ký services → `Build()` → middleware → `Map...` → `Run()`.
- Minimal API: endpoint là lambda, tham số tự bind từ route/query/body/DI.
- `TypedResults` + `Results<...>` cho kết quả rõ ràng; Problem Details cho lỗi.
- .NET 10 có validation có sẵn cho Minimal API.

Bài tập (có lời giải)

1. Thêm `PUT /api/notes/{id}` cập nhật ghi chú, trả 404 nếu không tồn tại.
2. Thêm endpoint filter cho cả group: thiếu header `X-Api-Key: secret` thì trả 401.
3. 404 phải trả về Problem Details có `title` rõ ràng bằng tiếng Việt.

▼ Lời giải

```
using System.Collections.Concurrent;
using System.Net.Http.Json;
using Microsoft.AspNetCore.Http.HttpResults;

var builder = WebApplication.CreateBuilder(args);
builder.Logging.SetMinimumLevel(LogLevel.Warning);
builder.WebHost.UseUrls("http://127.0.0.1:5198");
builder.Services.AddProblemDetails();
var app = builder.Build();

var notes = new ConcurrentDictionary<int, Note>();
notes[1] = new Note(1, "Học Minimal API");
```

```
// Bài 2: endpoint filter kiểm tra header X-API-Key cho cả group
var api = app.MapGroup("/api/notes").AddEndpointFilter(async (ctx, next) =>
    ctx.HttpContext.Request.Headers["X-API-Key"] == "secret"
    ? await next(ctx)
    : TypedResults.Problem(statusCode: 401, title: "Thiếu hoặc sai X-API-Key"));

// Bài 3: 404 trả về ProblemDetails có title rõ ràng
api.MapGet("/{id:int}", Results<Ok<Note>, ProblemHttpResult> (int id) =>
    notes.TryGetValue(id, out var n) ? TypedResults.Ok(n)
    : TypedResults.Problem(statusCode: 404, title: $"Không có ghi chú {id}"));

// Bài 1: PUT cập nhật, trả 404 nếu không có
api.MapPut("/{id:int}", Results<Ok<Note>, ProblemHttpResult> (int id, UpdateNote body) =>
{
    if (!notes.ContainsKey(id)) return TypedResults.Problem(statusCode: 404, title: $"Không có ghi chú {id}");
    var updated = new Note(id, body.Text);
    notes[id] = updated;
    return TypedResults.Ok(updated);
});

await app.StartAsync();
using var http = new HttpClient { BaseAddress = new Uri("http://127.0.0.1:5198") };

async Task Show(string label, HttpRequestMessage req, bool withKey = true)
{
    if (withKey) req.Headers.Add("X-API-Key", "secret");
    var res = await http.SendAsync(req);
    Console.WriteLine($"{label} -> {(int)res.StatusCode}: {await res.Content.ReadAsStringAsync()}");
}

await Show("Bài 2: GET không có key", new(HttpMethod.Get, "/api/notes/1"), withKey: false);
await Show("Bài 1: PUT /api/notes/1", new(HttpMethod.Put, "/api/notes/1") { Content =
JsonContent.Create(new { text = "Đã học xong" }) });
await Show("Bài 1: GET /api/notes/1", new(HttpMethod.Get, "/api/notes/1"));
await Show("Bài 3: GET /api/notes/7", new(HttpMethod.Get, "/api/notes/7"));
await app.StopAsync();

public record Note(int Id, string Text);
public record UpdateNote(string Text);
```

```
Bài 2: GET không có key -> 401: {"type":"https://tools.ietf.org/html/rfc9110#section-15.5.2","title":"Thiếu hoặc sai X-API-Key","status":401,"traceId":"00-16cf65097b7cca1d2e2200734da61303-5d8707481bc6e879-00"}
Bài 1: PUT /api/notes/1 -> 200: {"id":1,"text":"Đã học xong"}
Bài 1: GET /api/notes/1 -> 200: {"id":1,"text":"Đã học xong"}
Bài 3: GET /api/notes/7 -> 404: {"type":"https://tools.ietf.org/html/rfc9110#section-15.5.5","title":"Không có ghi chú 7","status":404,"traceId":"00-30e79e512b2f0af0ed2abc9f89f6234d-38ebeb3e4c6ed537-00"}
```

AddEndpointFilter trên MapGroup áp dụng cho mọi endpoint trong group. (API key trong ví dụ chỉ để học; hệ thống thật dùng JWT/OAuth và lưu bí mật trong cấu hình an toàn.)

Nguồn tham khảo (Sources)

- Minimal APIs overview (nguồn Microsoft Learn): <https://github.com/dotnet/AspNetCore.Docs/blob/main/aspnetcore/fundamentals/minimal-apis.md>
- Route handlers: <https://github.com/dotnet/AspNetCore.Docs/blob/main/aspnetcore/fundamentals/minimal-apis/route-handlers.md>

- Parameter binding: <https://github.com/dotnet/AspNetCore.Docs/blob/main/aspnetcore/fundamentals/minimal-apis/parameter-binding.md>
- Responses (TypedResults): <https://github.com/dotnet/AspNetCore.Docs/blob/main/aspnetcore/fundamentals/minimal-apis/responses.md>
- Validation support in Minimal APIs (.NET 10 release notes): <https://github.com/dotnet/AspNetCore.Docs/blob/main/aspnetcore/release-notes/aspnetcore-10/includes/ValidationSupportMinimalAPI.md>
- Middleware: <https://github.com/dotnet/AspNetCore.Docs/blob/main/aspnetcore/fundamentals/middleware/index.md>
- Handle errors in APIs (Problem Details): <https://github.com/dotnet/AspNetCore.Docs/blob/main/aspnetcore/fundamentals/error-handling-api.md>

Chương 2 — EF Core với SQLite

Mục tiêu

- Hiểu ORM và EF Core: `DbContext`, `DbSet<T>`, entity, navigation, foreign key.
- Viết query LINQ và xem SQL mà EF Core sinh ra.
- Hiểu *change tracking*, `AsNoTracking`, `Include`, projection bằng `Select`.
- Dùng `ExecuteUpdate`/`ExecuteDelete`, transaction.
- Dùng *migrations* với `dotnet ef` để quản lý schema.

Giải thích đơn giản

EF Core (Entity Framework Core) là ORM (Object-Relational Mapper): bạn làm việc với object C#, EF Core dịch sang SQL. Tương đương:

Node/TS	Java	.NET
Prisma / TypeORM	JPA / Hibernate	EF Core
<code>prisma.book.findMany({ where })</code>	<code>repository.findAll(spec)</code>	<code>db.Books.Where(...).ToListAsync()</code>
<code>prisma migrate</code>	Flyway / Liquibase	<code>dotnet ef migrations</code>

- `DbContext` = một "phiên làm việc" với database (giống `EntityManager` của JPA).
- `DbSet<Book>` = một bảng, query được bằng LINQ (Tập 2, Chương 3).
- Bộ sách dùng **SQLite** để chạy được ở mọi máy, không cần cài database server. Đổi sang SQL Server/PostgreSQL chủ yếu chỉ đổi provider (`UseSqlServer`, `UseNpgsql`).

Ví dụ

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="10.0.12" />
  </ItemGroup>

</Project>
```

Model và DbContext:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;

namespace V3Ch02.EfCore;

public class Author
{
    public int Id { get; set; }
    public required string Name { get; set; }
}
```

```

        public List<Book> Books { get; } = []; // navigation: 1 author - n books
    }

    public class Book
    {
        public int Id { get; set; }
        public required string Title { get; set; }
        public decimal Price { get; set; }
        public int Year { get; set; }
        public int AuthorId { get; set; } // foreign key
        public Author Author { get; set; } = null!; // navigation
    }

    public class LibraryContext(string connectionString) : DbContext
    {
        public DbSet<Author> Authors => Set<Author>();
        public DbSet<Book> Books => Set<Book>();

        /// <summary>When true, the SQL that EF Core sends to SQLite is printed.</summary>
        public static bool ShowSql { get; set; }

        protected override void OnConfiguring(DbContextOptionsBuilder options) => options
            .UseSqlite(connectionString)
            .LogTo(sql => { if (ShowSql) Console.WriteLine("    SQL> " + OneLine(sql)); },
                [DbLoggerCategory.Database.Command.Name], LogLevel.Information);

        protected override void OnModelCreating(ModelBuilder model)
        {
            model.Entity<Author>().Property(a => a.Name).HasMaxLength(100);
            model.Entity<Book>().Property(b => b.Title).HasMaxLength(200);
            model.Entity<Book>().HasIndex(b => b.Title);
            model.Entity<Book>().Property(b => b.Price).HasConversion<double>(); // SQLite has no
decimal type
        }

        private static string OneLine(string log)
        {
            // Keep only the SQL text (after the first blank line) on one line.
            var idx = log.IndexOf(Environment.NewLine, StringComparison.Ordinal);
            var sql = idx >= 0 ? log[(idx + Environment.NewLine.Length)..] : log;
            return string.Join(' ', sql.Split(Environment.NewLine,
StringSplitOptions.RemoveEmptyEntries).Select(s => s.Trim()));
        }
    }

```

Chương trình:

```

using Microsoft.EntityFrameworkCore;
using V3Ch02.EfCore;

string dbPath = Path.Combine(Path.GetTempPath(), "v3ch02-library.db");
File.Delete(dbPath);
string cs = $"Data Source={dbPath}";

// 1. Tạo database từ model (demo). Dự án thật dùng migrations (xem chương).
await using (var db = new LibraryContext(cs))
{
    await db.Database.EnsureCreatedAsync();
    Console.WriteLine($"Đã tạo DB: {Path.GetFileName(dbPath)}");
}

// 2. Thêm dữ liệu: Add + SaveChanges (1 transaction)
await using (var db = new LibraryContext(cs))
{
    var nam = new Author { Name = "Nguyễn Nhật Ánh" };
    nam.Books.Add(new Book { Title = "Mắt biếc", Price = 110_000m, Year = 1990 });
}

```

```

2008 });
    nam.Books.Add(new Book { Title = "Cho tôi xin một vé đi tuổi thơ", Price = 85_000m, Year =
    var to = new Author { Name = "Tô Hoài" });
    to.Books.Add(new Book { Title = "Đế Mèn phiêu lưu ký", Price = 60_000m, Year = 1941 });
    db.Authors.AddRange(nam, to);
    int rows = await db.SaveChangesAsync();
    Console.WriteLine($"SaveChanges: {rows} dòng; id tác giả = {nam.Id}, {to.Id}");
}

// 3. Query LINQ -> SQL
await using (var db = new LibraryContext(cs))
{
    LibraryContext.ShowSql = true;
    Console.WriteLine("Sách sau năm 1980, giá giảm dần:");
    var books = await db.Books
        .Where(b => b.Year > 1980)
        .OrderByDescending(b => b.Price)
        .Select(b => new { b.Title, b.Price, Author = b.Author.Name }) // projection: chỉ
        .ToListAsync();
    LibraryContext.ShowSql = false;
    foreach (var b in books) Console.WriteLine($" {b.Title} - {b.Author} - {b.Price:N0}");
}

// 4. Include (eager loading) và GroupBy
await using (var db = new LibraryContext(cs))
{
    var authors = await db.Authors.Include(a => a.Books).OrderBy(a => a.Name).ToListAsync();
    foreach (var a in authors) Console.WriteLine($" {a.Name}: {a.Books.Count} cuốn");

    var stats = await db.Books.GroupBy(b => b.Author.Name)
        .Select(g => new { Author = g.Key, Avg = g.Average(b => (double)b.Price) })
        .ToListAsync();
    foreach (var s in stats) Console.WriteLine($" Giá TB {s.Author}: {s.Avg:N0}");
}

// 5. Change tracking: sửa object rồi SaveChanges -> UPDATE
await using (var db = new LibraryContext(cs))
{
    var book = await db.Books.SingleAsync(b => b.Title == "Mất biếc");
    book.Price = 120_000m;
    Console.WriteLine($"Trạng thái trước SaveChanges: {db.Entry(book).State}");
    LibraryContext.ShowSql = true;
    await db.SaveChangesAsync();
    LibraryContext.ShowSql = false;
    Console.WriteLine($"Trạng thái sau SaveChanges: {db.Entry(book).State}");

    var readOnly = await db.Books.AsNoTracking().FirstAsync();
    Console.WriteLine($"AsNoTracking -> trạng thái: {db.Entry(readOnly).State}");
}

// 6. ExecuteUpdate / ExecuteDelete: cập nhật hàng loạt, không nạp entity
await using (var db = new LibraryContext(cs))
{
    LibraryContext.ShowSql = true;
    int updated = await db.Books.Where(b => b.Year < 2000)
        .ExecuteUpdateAsync(s => s.SetProperty(b => b.Price, b => b.Price * 0.9m));
    LibraryContext.ShowSql = false;
    Console.WriteLine($"ExecuteUpdate: giảm giá 10% cho {updated} cuốn");
}

// 7. Transaction thủ công: hoặc tất cả, hoặc không gì cả
await using (var db = new LibraryContext(cs))
{
    await using var tx = await db.Database.BeginTransactionAsync();
    try
    {

```

```

        db.Books.Add(new Book { Title = "Sách tạm", Price = 1m, Year = 2026, AuthorId = 1 });
        await db.SaveChangesAsync();
        db.Books.Add(new Book { Title = "Sách lỗi", Price = 1m, Year = 2026, AuthorId = 999
    }); // FK không tồn tại
        await db.SaveChangesAsync();
        await tx.CommitAsync();
    }
    catch (DbUpdateException ex)
    {
        await tx.RollbackAsync();
        Console.WriteLine($"Rollback vì: {ex.InnerException?.Message}");
    }
}

await using (var db = new LibraryContext(cs))
{
    Console.WriteLine($"Số sách cuối cùng: {await db.Books.CountAsync()} (không có 'Sách
tạm')");
}
File.Delete(dbPath);

```

Output thật (dòng SQL> là câu lệnh EF Core gửi tới SQLite):

```

Đã tạo DB: v3ch02-library.db
SaveChanges: 5 dòng; id tác giả = 1, 2
Sách sau năm 1980, giá giảm dần:
    SQL> Executed DbCommand (0ms) [Parameters=[], CommandType='Text', CommandTimeout='30'] SELECT
    "b"."Title", "b"."Price", "a"."Name" AS "Author" FROM "Books" AS "b" INNER JOIN "Authors" AS "a" ON
    "b"."AuthorId" = "a"."Id" WHERE "b"."Year" > 1980 ORDER BY "b"."Price" DESC
    Mất biếc - Nguyễn Nhật Ánh - 110,000
    Cho tôi xin một vé đi tuổi thơ - Nguyễn Nhật Ánh - 85,000
    Nguyễn Nhật Ánh: 2 cuốn
    Tô Hoài: 1 cuốn
    Giá TB Nguyễn Nhật Ánh: 97,500
    Giá TB Tô Hoài: 60,000
Trạng thái trước SaveChanges: Modified
    SQL> Executed DbCommand (0ms) [Parameters=[@p1='?' (DbType = Int32), @p0='?' (DbType = Double)],
    CommandType='Text', CommandTimeout='30'] UPDATE "Books" SET "Price" = @p0 WHERE "Id" = @p1 RETURNING
    1;
Trạng thái sau SaveChanges: Unchanged
AsNoTracking -> trạng thái: Detached
    SQL> Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30'] UPDATE
    "Books" AS "b" SET "Price" = "b"."Price" * 0.90000000000000002 WHERE "b"."Year" < 2000
ExecuteUpdate: giảm giá 10% cho 2 cuốn
Rollback vì: SQLite Error 19: 'FOREIGN KEY constraint failed'.
Số sách cuối cùng: 3 (không có 'Sách tạm')

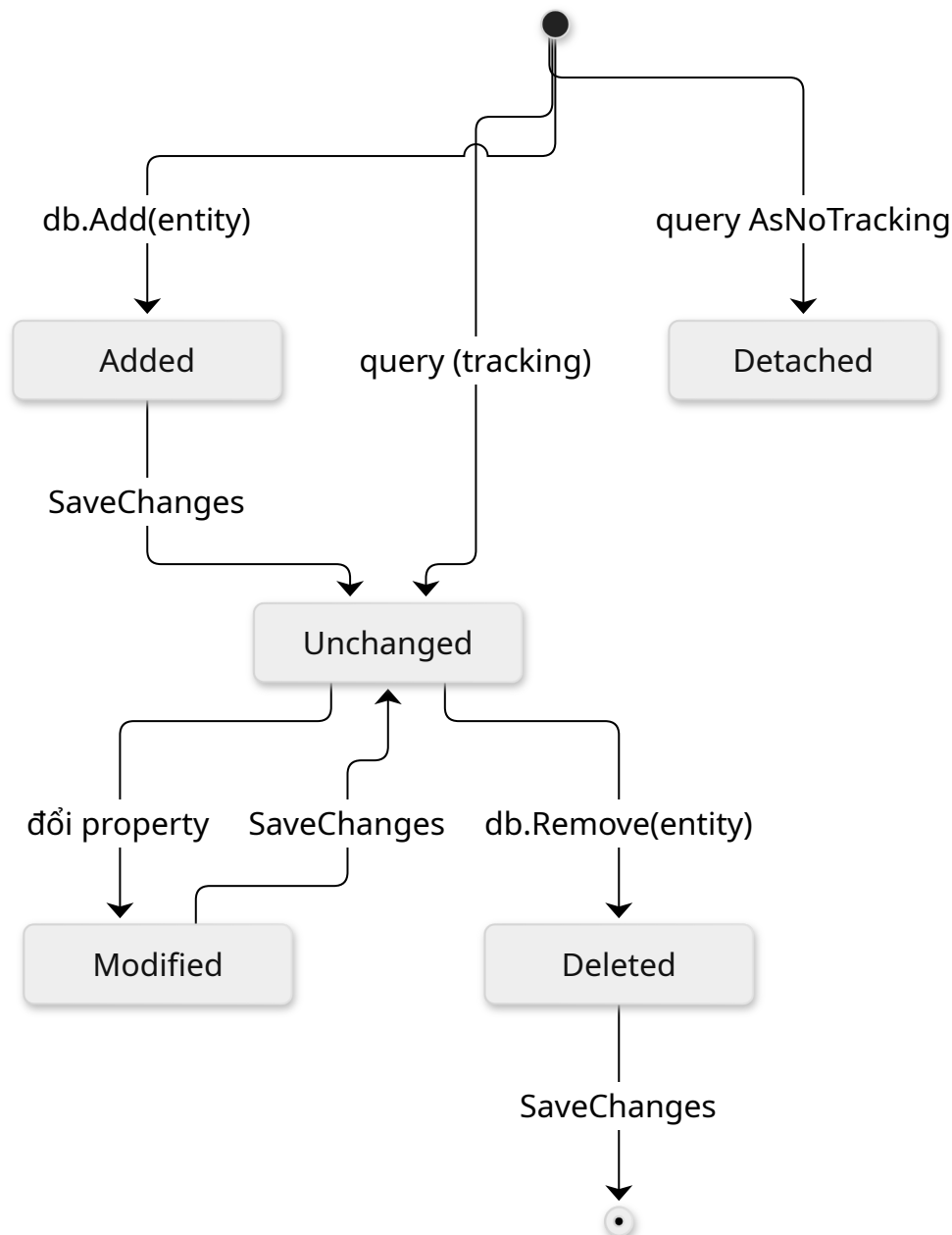
```

Để ý:

- Query có `Select(new { ... b.Author.Name })` → EF Core tự sinh `INNER JOIN` và chỉ lấy 3 cột.
- Sửa `book.Price` → trạng thái `Modified` → `SaveChanges` sinh đúng một câu `UPDATE` cho cột đó.
- `ExecuteUpdate` sinh một câu `UPDATE ... WHERE` chạy thẳng trên DB, không nạp entity.
- Giá trị `0.90000000000000002`: SQLite không có kiểu `decimal`, nên model chuyển `decimal` sang `double` → phép nhân bị sai số nhỏ. Đây là giới hạn đã được ghi trong tài liệu provider SQLite.
- Transaction: lỗi khóa ngoại ở lệnh thứ hai → `Rollback` → "Sách tạm" cũng không được lưu.

Đi sâu

Change tracking



- Query mặc định là **tracking**: EF Core nhớ giá trị gốc để biết cái gì đã đổi.
- Chỉ đọc (API GET) → dùng `AsNoTracking()` cho nhanh và nhẹ bộ nhớ.
- `SaveChanges` gom mọi thay đổi vào **một transaction**.

Nạp dữ liệu liên quan

Cách	Code	Khi nào
Eager loading	<code>.Include(a => a.Books)</code>	cần cả entity liên quan
Projection	<code>.Select(b => new Dto(b.Title, b.Author.Name))</code>	API trả DTO — nên dùng nhất
Lazy loading	cần cấu hình thêm	dễ gây N+1 query, bộ sách không dùng

N+1 query: lấy 100 tác giả (1 query) rồi với mỗi tác giả lại query sách (100 query). Tránh bằng `Include` hoặc `projection`.

Migrations

`EnsureCreated()` chỉ hợp cho demo/test: nó tạo schema từ model nhưng **không cập nhật** được khi model đổi. Dự án thật dùng **migrations**. Dự án cuối (`final-project/`) dùng công cụ `dotnet-ef 10.0.12` cài qua *local tool manifest* (`dotnet-tools.json`):

```
dotnet tool restore
dotnet ef migrations add InitialCreate --project src/TaskBoard.Infrastructure --startup-project
src/TaskBoard.Api --output-dir Migrations
```

Output thật của các lệnh kiểm tra:

```
$ dotnet ef migrations list
20260928145829_InitialCreate
$ dotnet ef migrations has-pending-model-changes
No changes have been made to the model since the last migration.
$ dotnet ef migrations script (first 14 lines)
CREATE TABLE IF NOT EXISTS "__EFMigrationsHistory" (
    "MigrationId" TEXT NOT NULL CONSTRAINT "PK__EFMigrationsHistory" PRIMARY KEY,
    "ProductVersion" TEXT NOT NULL
);

BEGIN TRANSACTION;
CREATE TABLE "tasks" (
    "Id" INTEGER NOT NULL CONSTRAINT "PK_tasks" PRIMARY KEY AUTOINCREMENT,
    "Title" TEXT NOT NULL,
    "Description" TEXT NULL,
    "Status" TEXT NOT NULL,
    "Priority" INTEGER NOT NULL,
    "DueDate" TEXT NULL,
    "CreatedAt" TEXT NOT NULL,
```

- `(Pending)` = migration chưa được áp dụng vào file DB trên máy (API sẽ áp dụng khi khởi động bằng `Database.Migrate()`).
- `has-pending-model-changes` nên chạy trong CI: nếu ai đó sửa model mà quên tạo migration, lệnh báo lỗi. Từ EF Core 9, `Migrate()` cũng ném exception nếu model có thay đổi chưa có migration.
- Khi thử `migrations script --idempotent`, SQLite báo *"Generating idempotent scripts for migrations is not currently supported for SQLite"* (lỗi thật khi viết chương) — với SQL Server/PostgreSQL thì dùng được.
- Áp dụng migration lúc khởi động là cách đơn giản; hệ thống lớn nên chạy migration như một bước deploy riêng (theo tài liệu EF Core).

Lỗi và bẫy thường gặp

- **DbContext không thread-safe**: không dùng một `DbContext` cho nhiều request/luồng. Trong ASP.NET Core, `AddDbContext` đăng ký **Scoped** (mỗi request một instance).
- **Quên await/dùng .Result** với `ToListAsync()` → chặn luồng.
- **N+1 query** khi duyệt navigation trong vòng lặp.

- **Kéo cả bảng về rồi mới lọc:** `db.Books.ToList().Where(...)` lọc trên RAM. Hãy `Where` trước `ToList`.
- **SQLite và `decimal/DateTimeOffset`:** không có kiểu tương ứng; không `ORDER BY` được `DateTimeOffset`. Dùng converter hoặc kiểu khác (tài liệu provider SQLite).
- **Lưu tiền bằng `double`:** sai số. Với SQLite có thể lưu số nguyên (đồng/xu) như Bài tập 1.
- **Enum lưu dạng chuỗi rồi `ORDER BY`:** sắp theo chữ cái, không theo giá trị. Dự án cuối lưu `Priority` dạng số vì lý do này.

Tóm tắt

- `DbContext` + `DbSet<T>` + LINQ = truy vấn database bằng C#.
- Tracking cho ghi, `AsNoTracking` + projection cho đọc.
- `ExecuteUpdate/Delete` cho thao tác hàng loạt; transaction cho nhiều bước.
- Migrations (dotnet ef) để quản lý schema theo thời gian.

Bài tập (có lời giải)

1. Với `Category` và `Product`, tìm danh mục có tổng giá sản phẩm cao nhất (một query).
2. Lấy trang 2 (mỗi trang 2 sản phẩm), sắp theo tên.
3. Cài *soft delete*: thêm `IsDeleted` và *global query filter* để query mặc định bỏ qua sản phẩm đã xóa; dùng `IgnoreQueryFilters()` để đếm tất cả.

▼ Lời giải

```
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;

// In-memory SQLite: DB sống khi connection còn mở (tiện cho demo và test)
await using var connection = new SqliteConnection("Data Source=:memory:");
await connection.OpenAsync();
var options = new DbContextOptionsBuilder<ShopContext>().UseSqlite(connection).Options;

await using (var db = new ShopContext(options))
{
    await db.Database.EnsureCreatedAsync();
    var fruits = new Category { Name = "Trái cây" };
    var drinks = new Category { Name = "Đồ uống" };
    db.Products.AddRange(
        new Product { Name = "Cam", Price = 30_000, Category = fruits },
        new Product { Name = "Xoài", Price = 45_000, Category = fruits },
        new Product { Name = "Trà sữa", Price = 35_000, Category = drinks },
        new Product { Name = "Cà phê", Price = 25_000, Category = drinks },
        new Product { Name = "Nước cam", Price = 20_000, Category = drinks });
    await db.SaveChangesAsync();
}

await using (var db = new ShopContext(options))
{
    // Bài 1: danh mục có tổng giá cao nhất
    var top = await db.Categories
        .Select(c => new { c.Name, Total = c.Products.Sum(p => p.Price) })
        .OrderByDescending(x => x.Total)
        .FirstAsync();
    Console.WriteLine($"Bài 1: {top.Name} (tổng {top.Total:N0})");

    // Bài 2: phân trang, trang 2, mỗi trang 2 sản phẩm, sắp theo tên
}
```

```

var page2 = await db.Products.OrderBy(p => p.Name).Skip(2).Take(2).Select(p =>
p.Name).ToListAsync();
Console.WriteLine($"Bài 2: trang 2 = {string.Join(", ", page2)}");

// Bài 3: soft delete với global query filter
var coffee = await db.Products.SingleAsync(p => p.Name == "Cà phê");
coffee.IsDeleted = true;
await db.SaveChangesAsync();
}

await using (var db = new ShopContext(options))
{
    Console.WriteLine($"Bài 3: số sản phẩm thấy được = {await db.Products.CountAsync()}");
    Console.WriteLine($"Bài 3: kể cả đã xóa mềm = {await
db.Products.IgnoreQueryFilters().CountAsync()}");
}

public class Category
{
    public int Id { get; set; }
    public required string Name { get; set; }
    public List<Product> Products { get; } = [];
}

public class Product
{
    public int Id { get; set; }
    public required string Name { get; set; }
    public long Price { get; set; } // lưu tiền VND dạng số nguyên (đồng)
    public bool IsDeleted { get; set; }
    public int CategoryId { get; set; }
    public Category Category { get; set; } = null!;
}

public class ShopContext(DbContextOptions<ShopContext> options) : DbContext(options)
{
    public DbSet<Product> Products => Set<Product>();
    public DbSet<Category> Categories => Set<Category>();

    protected override void OnModelCreating(ModelBuilder model) =>
        model.Entity<Product>().HasQueryFilter(p => !p.IsDeleted);
}

```

Bài 1: Đồ uống (tổng 80,000)
 Bài 2: trang 2 = Nước cam, Trà sữa
 Bài 3: số sản phẩm thấy được = 4
 Bài 3: kể cả đã xóa mềm = 5

Lời giải dùng SQLite **in-memory** (Data Source=:memory:): database tồn tại khi connection còn mở. Rất tiện cho test (dự án cuối dùng cách tương tự trong integration test).

Nguồn tham khảo (Sources)

- EF Core overview: <https://github.com/dotnet/EntityFramework.Docs/blob/main/entity-framework/core/index.md>
- Tracking vs. no-tracking queries: <https://github.com/dotnet/EntityFramework.Docs/blob/main/entity-framework/core/querying/tracking.md>
- Eager loading: <https://github.com/dotnet/EntityFramework.Docs/blob/main/entity-framework/core/querying/related-data/eager.md>

- ExecuteUpdate and ExecuteDelete: <https://github.com/dotnet/EntityFramework.Docs/blob/main/entity-framework/core/saving/execute-insert-update-delete.md>
- Transactions: <https://github.com/dotnet/EntityFramework.Docs/blob/main/entity-framework/core/saving/transactions.md>
- Migrations overview: <https://github.com/dotnet/EntityFramework.Docs/blob/main/entity-framework/core/managing-schemas/migrations/index.md>
- Applying migrations: <https://github.com/dotnet/EntityFramework.Docs/blob/main/entity-framework/core/managing-schemas/migrations/applying.md>
- SQLite provider limitations: <https://github.com/dotnet/EntityFramework.Docs/blob/main/entity-framework/core/providers/sqlite/limitations.md>
- Global query filters: <https://github.com/dotnet/EntityFramework.Docs/blob/main/entity-framework/core/querying/filters.md>

Chương 3 — Hiệu năng: Span, bộ nhớ, BenchmarkDotNet

Mục tiêu

- Hiểu cách .NET quản lý bộ nhớ: stack, heap, GC (Garbage Collector — bộ gom rác) và các thế hệ (generation).
- Dùng `Span<T>`, `ReadOnlySpan<T>`, `stackalloc`, `ArrayPool<T>` để giảm cấp phát.
- Đo hiệu năng **đúng cách** bằng BenchmarkDotNet, đọc bảng kết quả.

Giải thích đơn giản

Mỗi lần bạn tạo object (chuỗi, mảng, class), .NET cấp phát trên **heap**. GC định kỳ dọn các object không còn dùng. GC nhanh, nhưng **cấp phát càng ít thì GC càng ít phải chạy** → ứng dụng mượt hơn, nhất là Web API xử lý hàng nghìn request/giây.

`Span<T>` là "cửa sổ" nhìn vào một vùng nhớ **có sẵn** (mảng, chuỗi, stack) mà không copy. Giống `subarray()` của `Uint8Array` trong JavaScript: cắt mà không tạo dữ liệu mới.

Quy tắc vàng: **đo trước, tối ưu sau**. Đừng đoán; dùng BenchmarkDotNet hoặc profiler.

Ví dụ

Ví dụ 1 — Span và cấp phát bộ nhớ

```
using System.Buffers;
using System.Runtime.InteropServices;

const string line = "SKU-001,Bàn phím cơ,1200000,15";

// 1. Span<T> là "cửa sổ" nhìn vào bộ nhớ có sẵn, không copy
int[] numbers = [10, 20, 30, 40, 50];
Span<int> middle = numbers.AsSpan(1, 3); // 20, 30, 40
middle[0] = 99; // sửa luôn mảng gốc
Console.WriteLine($"numbers = [{string.Join(", ", numbers)}], middle.Length = {middle.Length}");

// 2. ReadOnlySpan<char>: cắt chuỗi không tạo chuỗi mới
ReadOnlySpan<char> span = line;
int firstComma = span.IndexOf(',');
ReadOnlySpan<char> sku = span[..firstComma];
Console.WriteLine($"SKU = {sku.ToString()} (chỉ ToString() khi cần mới cấp phát)");

// 3. So sánh cấp phát bộ nhớ: string.Split vs Span
const int N = 100_000;
long before = GC.GetAllocatedBytesForCurrentThread();
long sum1 = 0;
for (int i = 0; i < N; i++) sum1 += ParseWithSplit(line);
long splitBytes = GC.GetAllocatedBytesForCurrentThread() - before;

before = GC.GetAllocatedBytesForCurrentThread();
long sum2 = 0;
for (int i = 0; i < N; i++) sum2 += ParseWithSpan(line);
```

```

long spanBytes = GC.GetAllocatedBytesForCurrentThread() - before;

Console.WriteLine($"Split: tổng={sum1}, cấp phát ~{splitBytes / N} bytes/lần");
Console.WriteLine($"Span : tổng={sum2}, cấp phát ~{spanBytes / N} bytes/lần");

// 4. stackalloc: bộ nhớ tạm trên stack (nhỏ, ngắn hạn), không cần GC
Span<byte> buffer = stackalloc byte[16];
Random.Shared.NextBytes(buffer);
Console.WriteLine($"stackalloc 16 bytes, Length = {buffer.Length}");

// 5. ArrayPool: mượn mảng lớn rồi trả lại, tránh cấp phát lặp lại
byte[] rented = ArrayPool<byte>.Shared.Rent(4096);
try
{
    Console.WriteLine($"Rent(4096) -> mảng dài {rented.Length} (có thể lớn hơn yêu cầu)");
}
finally
{
    ArrayPool<byte>.Shared.Return(rented);
}

// 6. C# 14 first-class span: mảng tự chuyển sang ReadOnlySpan khi gọi hàm
Console.WriteLine($"Sum(array) = {Sum(numbers)}");

// 7. CollectionsMarshal.AsSpan: duyệt List<T> không kiểm tra version
List<int> list = [1, 2, 3, 4];
foreach (ref int x in CollectionsMarshal.AsSpan(list)) x *= 10;
Console.WriteLine($"list = [{string.Join(", ", list)}]");

// 8. Thông tin GC
Console.WriteLine($"GC: gen0={GC.CollectionCount(0)}, gen1={GC.CollectionCount(1)}, gen2={GC.CollectionCount(2)}, server GC={System.Runtime.GCSettings.IsServerGC}");

static int ParseWithSplit(string csv)
{
    string[] parts = csv.Split(','); // cấp phát mảng + 4 chuỗi
    return int.Parse(parts[3]);
}

static int ParseWithSpan(ReadOnlySpan<char> csv)
{
    int last = csv.LastIndexOf(',');
    return int.Parse(csv[(last + 1)..]); // không cấp phát
}

static int Sum(ReadOnlySpan<int> values)
{
    int total = 0;
    foreach (var v in values) total += v;
    return total;
}

```

Output thật:

```

numbers = [10, 99, 30, 40, 50], middle.Length = 3
SKU = SKU-001 (chỉ ToString() khi cần mới cấp phát)
Split: tổng=1500000, cấp phát ~216 bytes/lần
Span : tổng=1500000, cấp phát ~0 bytes/lần
stackalloc 16 bytes, Length = 16
Rent(4096) -> mảng dài 4096 (có thể lớn hơn yêu cầu)
Sum(array) = 229
list = [10, 20, 30, 40]
GC: gen0=0, gen1=0, gen2=0, server GC=False

```

`string.Split` cấp phát khoảng 216 bytes mỗi lần (một mảng + 4 chuỗi). Bản dùng `Span` cấp phát 0 byte — cùng kết quả.

Ví dụ 2 — BenchmarkDotNet

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <!-- BenchmarkDotNet generates code; keep warnings as warnings for this project. -->
    <TreatWarningsAsErrors>>false</TreatWarningsAsErrors>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="BenchmarkDotNet" Version="0.15.8" />
  </ItemGroup>

</Project>
```

```
using System.Text;
using BenchmarkDotNet.Attributes;
using BenchmarkDotNet.Running;

// Chạy: dotnet run -c Release -- --job short --filter "*"
BenchmarkSwitcher.FromAssembly(typeof(ParsingBenchmarks).Assembly).Run(args);

[MemoryDiagnoser] // đo thêm bộ nhớ cấp phát và số lần GC
public class ParsingBenchmarks
{
    private const string Line = "SKU-001,Bàn phím cơ,1200000,15";

    [Benchmark(Baseline = true)]
    public int Split() => int.Parse(Line.Split(',')[3]);

    [Benchmark]
    public int Span()
    {
        ReadOnlySpan<char> s = Line;
        return int.Parse(s[(s.LastIndexOf(',') + 1)..]);
    }
}

[MemoryDiagnoser]
public class ConcatBenchmarks
{
    [Params(10, 1000)]
    public int Count { get; set; }

    [Benchmark(Baseline = true)]
    public string PlusEquals()
    {
        string s = "";
        for (int i = 0; i < Count; i++) s += i;
        return s;
    }

    [Benchmark]
    public string Builder()
    {
        var sb = new StringBuilder();
        for (int i = 0; i < Count; i++) sb.Append(i);
        return sb.ToString();
    }
}
```

Chạy với **ShortRun** (3 lần đo, nhanh nhưng kém chính xác hơn job mặc định). Output thật (chỉ giữ phần Summary; thông tin máy nằm ở đầu mỗi bảng):

```
$ dotnet run -c Release -- --job short --filter "*"
// * Summary *
BenchmarkDotNet v0.15.8, Linux Ubuntu 24.04.4 LTS (Noble Numbat)
Intel Xeon Processor 2.10GHz, 1 CPU, 4 logical and 4 physical cores
.NET SDK 10.0.112
[Host] : .NET 10.0.12 (10.0.12, 10.0.1226.42308), X64 RyuJIT x86-64-v4
ShortRun : .NET 10.0.12 (10.0.12, 10.0.1226.42308), X64 RyuJIT x86-64-v4
Job=ShortRun IterationCount=3 LaunchCount=1
WarmupCount=3
| Method | Count | Mean | Error | StdDev | Ratio | RatioSD | Gen0 |
Gen1 | Allocated | Alloc Ratio |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| PlusEquals | 10 | 145.40 ns | 58.83 ns | 3.224 ns | 1.00 | 0.03 | 0.0024 |
- | 336 B | 1.00 |
| Builder | 10 | 65.92 ns | 44.83 ns | 2.457 ns | 0.45 | 0.02 | 0.0011 |
- | 152 B | 0.45 |
| PlusEquals | 1000 | 270,880.19 ns | 249,518.68 ns | 13,676.961 ns | 1.00 | 0.06 | 20.7520 |
0.2441 | 2840456 B | 1.000 |
| Builder | 1000 | 6,575.39 ns | 3,711.97 ns | 203.465 ns | 0.02 | 0.00 | 0.1068 |
- | 14712 B | 0.005 |
// * Summary *
BenchmarkDotNet v0.15.8, Linux Ubuntu 24.04.4 LTS (Noble Numbat)
Intel Xeon Processor 2.10GHz, 1 CPU, 4 logical and 4 physical cores
.NET SDK 10.0.112
[Host] : .NET 10.0.12 (10.0.12, 10.0.1226.42308), X64 RyuJIT x86-64-v4
ShortRun : .NET 10.0.12 (10.0.12, 10.0.1226.42308), X64 RyuJIT x86-64-v4
Job=ShortRun IterationCount=3 LaunchCount=1
WarmupCount=3
| Method | Mean | Error | StdDev | Ratio | RatioSD | Gen0 | Allocated | Alloc Ratio |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| Split | 87.398 ns | 77.060 ns | 4.2239 ns | 1.00 | 0.06 | 0.0015 | 216 B | 1.00 |
| Span | 6.074 ns | 4.699 ns | 0.2575 ns | 0.07 | 0.00 | - | - | 0.00 |
Global total time: 00:01:00 (60.48 sec), executed benchmarks: 6
Load average khi chạy xong (1/5/15 phút): 6.21 4.85 4.65
```

Lưu ý về độ tin cậy: máy chạy là máy ảo dùng chung với các tiến trình khác (xem dòng load average). Cột **Error** lớn (với **Split**, Error gần bằng chính Mean) cho thấy nhiễu cao. Tỷ lệ (**Ratio**) và bộ nhớ (**Allocated**) đáng tin hơn thời gian tuyệt đối. Muốn số liệu chính xác, chạy job mặc định (không `--job short`) trên máy rảnh.

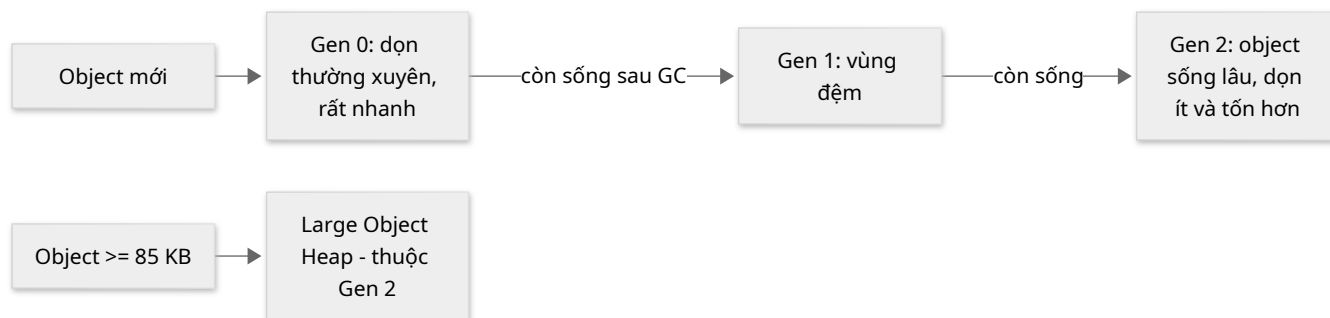
Đi sâu

Đọc bảng BenchmarkDotNet

Cột	Ý nghĩa
Mean	thời gian trung bình mỗi lần gọi
Error / StdDev	độ bất định / độ lệch chuẩn — càng nhỏ càng tin cậy
Ratio	so với phương thức <code>Baseline = true</code>
Gen0 / Gen1	số lần GC mỗi 1000 lần gọi
Allocated	bộ nhớ cấp phát mỗi lần gọi

Kết quả: `Span` nhanh hơn `Split` hơn 10 lần (Ratio khoảng 0.06–0.07 ở các lần chạy khi viết sách) và không cấp phát. Với `Count = 1000`, `+=` cấp phát ~2.8 MB (mỗi lần nối tạo chuỗi mới), `StringBuilder` chỉ ~14 KB.

GC và các thế hệ



- Phần lớn object "chết trẻ" → Gen 0 dọn rất rẻ.
- Object lớn (mảng ≥ 85.000 byte) vào LOH, dọn tốn kém → dùng `ArrayPool` để tái sử dụng.
- ASP.NET Core mặc định dùng `Server GC` (nhiều heap, tối ưu thông lượng); app console dùng `Workstation GC` (output `server GC=False`).

Công cụ giảm cấp phát

Công cụ	Dùng khi
<code>ReadOnlySpan<char></code>	parse/cắt chuỗi không tạo chuỗi con
<code>stackalloc</code>	buffer nhỏ (vài trăm byte), ngắn hạn, trong một method
<code>ArrayPool<T>.Shared</code>	buffer lớn dùng lặp lại (đọc stream, xử lý file)
<code>StringBuilder</code> / <code>string.Create</code>	ghép chuỗi nhiều phần
<code>ValueTask<T></code>	async thường hoàn thành đồng bộ (cache hit)
struct nhỏ, readonly struct	dữ liệu nhỏ, tránh cấp phát heap

C# 14 thêm *first-class span*: mảng tự chuyển sang `Span<T>` / `ReadOnlySpan<T>` khi gọi hàm, làm API dùng span dễ dùng hơn (`Sum(numbers)` trong ví dụ).

Giới hạn của Span

`Span<T>` là `ref struct`: chỉ sống trên stack. Không lưu vào field của class, không dùng qua `await` trong cùng biểu thức, không đưa vào lambda bất biến. Cần lưu lâu → dùng `Memory<T>`.

Lỗi và bẫy thường gặp

- **Benchmark bằng Stopwatch trong Debug**: sai lệch lớn (JIT chưa tối ưu, không warmup). Luôn chạy `BenchmarkDotNet` với `-c Release`.
- **Tối ưu code không nóng**: 90% thời gian thường nằm ở 10% code (và I/O, database). Đo trước bằng profiler (`dotnet-trace`, `dotnet-counters`).
- **stackalloc quá lớn** → `StackOverflowException` (không bắt được). Giữ dưới vài KB, hoặc dùng `ArrayPool` khi kích thước lớn/không biết trước.

- Quên Return mảng về ArrayPool → mất lợi ích; dùng try/finally.
- Dùng mảng từ pool sau khi đã Return → dữ liệu bị ghi đè bởi nơi khác.
- Tin số liệu từ máy ồn: xem cột Error/StdDev; so sánh Ratio thay vì thời gian tuyệt đối.

Tóm tắt

- Ít cấp phát = ít GC = nhanh và ổn định hơn.
- Span<T> cắt dữ liệu không copy; stackalloc, ArrayPool cho buffer.
- BenchmarkDotNet + [MemoryDiagnoser] là cách đo chuẩn; đọc Ratio và Allocated.

Bài tập (có lời giải)

1. Đếm số từ trong một câu mà **không cấp phát** (không dùng Split).
2. Tạo mã đơn hàng dạng ORD-2026-000042 vào buffer stackalloc bằng TryFormat.
3. Tính tổng các số trong chuỗi CSV 1000 phần tử, dùng ArrayPool và MemoryExtensions.Split.

▼ Lời giải

```
using System.Buffers;

// Bài 1: đếm số từ trong câu mà không cấp phát (không Split)
const string text = " Span giúp xử lý chuỗi nhanh và ít rác ";
long before = GC.GetAllocatedBytesForCurrentThread();
int words = CountWords(text);
long allocated = GC.GetAllocatedBytesForCurrentThread() - before;
Console.WriteLine($"Bài 1: {words} từ, cấp phát {allocated} bytes");

// Bài 2: định dạng mã đơn hàng vào buffer stackalloc bằng TryFormat
Span<char> buffer = stackalloc char[32];
int written = FormatOrderCode(buffer, 2026, 42);
Console.WriteLine($"Bài 2: {buffer[..written].ToString()}");

// Bài 3: tổng các số trong một chuỗi CSV dài dùng buffer mượn từ ArrayPool
string csv = string.Join(",", Enumerable.Range(1, 1000));
Console.WriteLine($"Bài 3: tổng = {SumCsv(csv)}");

static int CountWords(ReadOnlySpan<char> s)
{
    int count = 0;
    bool inWord = false;
    foreach (char c in s)
    {
        if (char.IsWhiteSpace(c)) inWord = false;
        else if (!inWord) { inWord = true; count++; }
    }
    return count;
}

static int FormatOrderCode(Span<char> destination, int year, int number)
{
    "ORD-".CopyTo(destination);
    int pos = 4;
    year.TryFormat(destination[pos..], out int w1); pos += w1;
    destination[pos++] = '-';
    number.TryFormat(destination[pos..], out int w2, "D6"); pos += w2;
    return pos;
}

static long SumCsv(string csv)
```

```

{
    int[] values = ArrayPool<int>.Shared.Rent(csv.Length / 2 + 1);
    try
    {
        int n = 0;
        foreach (var range in csv.AsSpan().Split(',')) // MemoryExtensions.Split
        {
            values[n++] = int.Parse(csv.AsSpan()[range]);
            long sum = 0;
            foreach (var v in values.AsSpan(0, n)) sum += v;
            return sum;
        }
    }
    finally
    {
        ArrayPool<int>.Shared.Return(values);
    }
}

```

Bài 1: 9 từ, cấp phát 0 bytes

Bài 2: ORD-2026-000042

Bài 3: tổng = 500500

Bài 1 đo bằng `GC.GetAllocatedBytesForCurrentThread()` và thấy đúng 0 byte.

Nguồn tham khảo (Sources)

- Memory and spans: <https://github.com/dotnet/docs/blob/main/docs/standard/memory-and-spans/index.md>
- Memory and Span usage guidelines: <https://github.com/dotnet/docs/blob/main/docs/standard/memory-and-spans/memory-t-usage-guidelines.md>
- Garbage collection fundamentals: <https://github.com/dotnet/docs/blob/main/docs/standard/garbage-collection/fundamentals.md>
- stackalloc: <https://github.com/dotnet/docs/blob/main/docs/csharp/language-reference/operator/stackalloc.md>
- First-class Span types (C# 14): <https://github.com/dotnet/csharplang/blob/main/proposals/csharp-14.0/first-class-span-types.md>
- BenchmarkDotNet repository: <https://github.com/dotnet/BenchmarkDotNet>
- BenchmarkDotNet on NuGet: <https://www.nuget.org/packages/BenchmarkDotNet/0.15.8>

Chương 4 — Concurrency (xử lý đồng thời)

Mục tiêu

- Phân biệt *concurrency* (đồng thời), *parallelism* (song song) và *asynchrony* (bất đồng bộ).
- Nhận ra *race condition* và sửa bằng `lock` (`System.Threading.Lock`), `Interlocked`, `collection` đồng thời.
- Dùng `Task.Run`, `Parallel.For/ForEach/ForEachAsync`, `PLINQ` cho việc nặng CPU.
- Xây producer/consumer bằng `Channel<T>`.
- Tránh deadlock.

Giải thích đơn giản

- **Asynchrony** (Tập 2, Chương 5): chờ I/O mà không chặn luồng. Phù hợp gọi DB, HTTP, file.
- **Parallelism**: chia việc **tính toán** cho nhiều lõi CPU chạy cùng lúc.
- **Concurrency**: nhiều việc "đang diễn ra" cùng lúc (có thể song song hoặc xen kẽ).

Node.js chạy JavaScript trên một luồng, nên bạn hiếm khi gặp *race condition* trên biến. Java và .NET có **nhiều luồng thật** dùng chung bộ nhớ → phải đồng bộ khi nhiều luồng cùng sửa một dữ liệu. Web API ASP.NET Core xử lý nhiều request **song song** trên thread pool, nên mọi singleton phải an toàn đa luồng (thread-safe).

Ví dụ

```
using System.Collections.Concurrent;
using System.Diagnostics;
using System.Threading.Channels;

Console.WriteLine($"Số CPU logic: {Environment.ProcessorCount}");
const int N = 1_000_000;

// 1. Race condition: nhiều luồng cùng sửa một biến
int unsafeCounter = 0;
Parallel.For(0, N, _ => unsafeCounter++); // ++ không phải thao tác nguyên tử
Console.WriteLine($"Không đồng bộ: {unsafeCounter:N0} (mong đợi {N:N0})");

// 2. lock với System.Threading.Lock (C# 13 / .NET 9)
var gate = new Lock();
int lockedCounter = 0;
Parallel.For(0, N, _ => { lock (gate) { lockedCounter++; } });
Console.WriteLine($"lock: {lockedCounter:N0}");

// 3. Interlocked: nguyên tử, nhanh cho phép toán đơn giản
int atomicCounter = 0;
Parallel.For(0, N, _ => Interlocked.Increment(ref atomicCounter));
Console.WriteLine($"Interlocked: {atomicCounter:N0}");

// 4. ConcurrentDictionary: collection an toàn đa luồng
var wordCounts = new ConcurrentDictionary<string, int>();
string[] words = ["c#", "java", "c#", "ts", "c#", "java"];
Parallel.ForEach(Enumerable.Range(0, 1000), i =>
    wordCounts.AddOrUpdate(words[i % words.Length], 1, (_, old) => old + 1));
Console.WriteLine("ConcurrentDictionary: " + string.Join(", ", wordCounts.OrderBy(kv =>
    kv.Key).Select(kv => $"{kv.Key}={kv.Value}")));
```

```

// 5. Việc nặng CPU: tuần tự vs Task.Run vs PLINQ
static bool IsPrime(int n)
{
    if (n < 2) return false;
    for (int d = 2; d * d <= n; d++) if (n % d == 0) return false;
    return true;
}

const int Limit = 3_000_000;
var sw = Stopwatch.StartNew();
int seq = Enumerable.Range(0, Limit).Count(IsPrime);
var seqMs = sw.ElapsedMilliseconds;

sw.Restart();
// Chia đều theo kiểu "xen kẽ" (i, i+4, i+8...) để 4 task có lượng việc gần bằng nhau
var tasks = Enumerable.Range(0, 4).Select(offset => Task.Run(() =>
{
    int count = 0;
    for (int n = offset; n < Limit; n += 4) if (IsPrime(n)) count++;
    return count;
}));
int par = (await Task.WhenAll(tasks)).Sum();
var parMs = sw.ElapsedMilliseconds;

sw.Restart();
int plinq = Enumerable.Range(0, Limit).AsParallel().Count(IsPrime);
var plinqMs = sw.ElapsedMilliseconds;
Console.WriteLine($"Số nguyên tố < 3 triệu: tuần tự={seq:N0} ({seqMs} ms), 4 task={par:N0} ({parMs} ms), PLINQ={plinq:N0} ({plinqMs} ms)");

// 6. Parallel.ForEachAsync: giới hạn số việc I/O chạy cùng lúc
int inFlight = 0, maxInFlight = 0;
await Parallel.ForEachAsync(Enumerable.Range(1, 12), new ParallelOptions {
    MaxDegreeOfParallelism = 3 }, async (i, ct) =>
{
    int now = Interlocked.Increment(ref inFlight);
    int seen;
    while (now > (seen = Volatile.Read(ref maxInFlight)) && Interlocked.CompareExchange(ref
maxInFlight, now, seen) != seen) {
        await Task.Delay(30, ct);
        Interlocked.Decrement(ref inFlight);
    }
});
Console.WriteLine($"ForEachAsync: tối đa {maxInFlight} việc cùng lúc (giới hạn 3)");

// 7. Channel<T>: producer / consumer có giới hạn (back-pressure)
var channel = Channel.CreateBounded<int>(new BoundedChannelOptions(capacity: 5)
{
    FullMode = BoundedChannelFullMode.Wait,
});
var producer = Task.Run(async () =>
{
    for (int i = 1; i <= 20; i++) await channel.Writer.WriteAsync(i);
    channel.Writer.Complete();
});
var consumers = Enumerable.Range(1, 2).Select(id => Task.Run(async () =>
{
    int handled = 0, total = 0;
    await foreach (var item in channel.Reader.ReadAllAsync())
    {
        handled++;
        total += item;
        await Task.Delay(5);
    }
    return (id, handled, total);
})).ToArray();
await producer;
var results = await Task.WhenAll(consumers);

```

```

        Console.WriteLine($"Channel: tổng xử lý = {results.Sum(r => r.handled)} việc, tổng giá trị = {results.Sum(r => r.total)} " +
            $"(consumer 1 làm {results[0].handled}, consumer 2 làm {results[1].handled})");

        // Máy build dùng chung với tiến trình khác? Load average cao làm song song kém hiệu quả.
        if (File.Exists("/proc/loadavg"))
            Console.WriteLine($"Load average (1/5/15 phút): {string.Join(" ",
                File.ReadAllText("/proc/loadavg").Split(' ')[..3])}");

```

Output thật (các con số thời gian và giá trị "Không đồng bộ" thay đổi mỗi lần chạy):

```

Số CPU logic: 4
Không đồng bộ: 549,063 (mong đợi 1,000,000)
lock: 1,000,000
Interlocked: 1,000,000
ConcurrentDictionary: c#=500, java=333, ts=167
Số nguyên tố < 3 triệu: tuần tự=216,816 (766 ms), 4 task=216,816 (454 ms), PLINQ=216,816 (481 ms)
ForEachAsync: tối đa 3 việc cùng lúc (giới hạn 3)
Channel: tổng xử lý = 20 việc, tổng giá trị = 210 (consumer 1 làm 10, consumer 2 làm 10)
Load average (1/5/15 phút): 2.94 3.71 4.28

```

Đọc kết quả:

- **Race condition:** unsafeCounter++ gồm 3 bước (đọc, cộng, ghi). Hai luồng đọc cùng giá trị rồi cùng ghi → mất lượt cộng. Kết quả nhỏ hơn 1.000.000 và khác nhau mỗi lần chạy.
- lock và Interlocked cho kết quả đúng.
- Phần số nguyên tố: trên máy rảnh 4 lõi, bản song song thường nhanh hơn rõ rệt. Khi viết sách, máy build là máy ảo 4 lõi **dùng chung** với nhiều tiến trình khác, nên mức tăng tốc đo được thấp và thay đổi giữa các lần chạy (xem load average ở dòng cuối: số càng gần hoặc vượt số lõi thì các luồng càng phải tranh CPU). Bài học: **luôn đo trên môi trường giống production**.

Đi sâu

Chọn công cụ đồng bộ

Công cụ	Dùng khi	Ghi chú
Interlocked	tăng/giảm/đổi một biến số	nhanh nhất, không khóa
lock (Lock)	bảo vệ nhiều lệnh (critical section)	C# 13 + .NET 9: kiểu System.Threading.Lock riêng
SemaphoreSlim	giới hạn số luồng/tác vụ vào cùng lúc; dùng được với await	WaitAsync()
ConcurrentDictionary, ConcurrentQueue	collection dùng chung	không cần tự lock
Channel<T>	producer/consumer bất đồng bộ	có back-pressure (bounded)
Immutable (record, ImmutableList)	chia sẻ dữ liệu chỉ đọc	không cần khóa

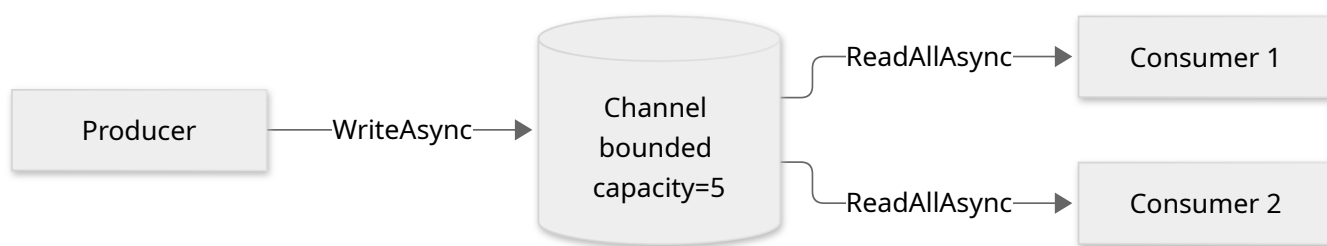
Không await bên trong lock (compiler báo lỗi). Cần khóa quanh code async → dùng SemaphoreSlim.WaitAsync().

Parallel vs Task.Run vs PLINQ

- `Parallel.For/ForEach`: vòng lặp CPU song song, tự chia việc.
- `Parallel.ForEachAsync` (.NET 6): nhiều tác vụ **async** với `MaxDegreeOfParallelism` — ví dụ gọi 1000 URL nhưng tối đa 10 cùng lúc.
- `Task.Run`: đẩy một việc CPU ra thread pool (ví dụ: tránh chặn luồng xử lý request).
- PLINQ `.AsParallel()`: LINQ song song cho dữ liệu lớn, tính toán nặng.

Trong ASP.NET Core, **không** bọc code I/O bằng `Task.Run` — đã có `async/await`. Chỉ dùng `Task.Run` cho việc CPU thật sự nặng.

Channel — producer/consumer



Khi channel đầy (bounded), `WriteAsync` **chờ** → producer tự chậm lại theo tốc độ consumer (*back-pressure*). Mẫu này thường dùng cho hàng đợi việc nền (`BackgroundService`) trong Web API.

Deadlock

Luồng A giữ khóa X chờ khóa Y; luồng B giữ Y chờ X → cả hai chờ mãi. Cách tránh: **luôn lấy khóa theo cùng một thứ tự** (Bài tập 1), giữ khóa trong thời gian ngắn nhất, không gọi code lạ (callback, event) khi đang giữ khóa.

Lỗi và bẫy thường gặp

- Dùng `List<T>/Dictionary` chung giữa các request mà không khóa → dữ liệu hỏng, exception khó tái hiện.
- **Singleton có trạng thái thay đổi** trong Web API mà không thread-safe.
- `Parallel.ForEach` với code **async** (`async lambda`) → thành `async void`, không chờ được. Dùng `Parallel.ForEachAsync`.
- **Song song hóa việc quá nhỏ** → chi phí chia việc lớn hơn lợi ích.
- `lock(this)` hoặc **lock trên chuỗi** → code khác có thể khóa cùng object. Dùng field `private readonly Lock`.
- **Tin rằng song song luôn nhanh hơn**: phụ thuộc số lỗi rảnh, cách chia việc, bộ nhớ.

Tóm tắt

- Nhiều luồng sửa chung dữ liệu → cần `Interlocked`, `lock`, hoặc collection đồng thời.
- CPU nặng: `Parallel`, PLINQ, `Task.Run`; I/O: `async/await`, `Parallel.ForEachAsync`, `SemaphoreSlim`.
- `Channel<T>` cho producer/consumer có back-pressure.

- Khóa theo thứ tự cố định để tránh deadlock.

Bài tập (có lời giải)

1. Viết `Account.Transfer(from, to, amount)` an toàn đa luồng, không deadlock khi hai luồng chuyển ngược chiều nhau; chạy 10.000 lần chuyển song song và kiểm tra tổng tiền không đổi.
2. Tạo pipeline 2 bước bằng `Channel<T>`: bước 1 đưa tên vào, bước 2 viết hoa, cuối cùng in kết quả.
3. Viết `RequestCounter` đếm số request theo đường dẫn, an toàn khi gọi từ nhiều luồng.

▼ Lời giải

```
using System.Threading.Channels;

// Bài 1: chuyển tiền an toàn giữa 2 tài khoản, tránh deadlock bằng thứ tự lock theo Id
var a = new Account(1, 1_000);
var b = new Account(2, 1_000);
var transfers = Enumerable.Range(0, 10_000).Select(i => Task.Run(() =>
{
    if (i % 2 == 0) Account.Transfer(a, b, 1); else Account.Transfer(b, a, 1);
}));
await Task.WhenAll(transfers);
Console.WriteLine($"Bài 1: a={a.Balance}, b={b.Balance}, tổng={a.Balance + b.Balance} (luôn 2000)");

// Bài 2: pipeline 2 bước bằng Channel: đọc -> xử lý -> ghi
var raw = Channel.CreateUnbounded<string>();
var processed = Channel.CreateBounded<string>(10);
var stage1 = Task.Run(async () =>
{
    foreach (var line in new[] { "an", "binh", "chi", "dung" }) await
raw.Writer.WriteAsync(line);
    raw.Writer.Complete();
});
var stage2 = Task.Run(async () =>
{
    await foreach (var s in raw.Reader.ReadAllAsync()) await
processed.Writer.WriteAsync(s.ToUpperInvariant());
    processed.Writer.Complete();
});
var output = new List<string>();
await foreach (var s in processed.Reader.ReadAllAsync()) output.Add(s);
await Task.WhenAll(stage1, stage2);
Console.WriteLine($"Bài 2: {string.Join(" -> ", output)}");

// Bài 3: bộ đếm request theo endpoint an toàn đa luồng
var counter = new RequestCounter();
Parallel.For(0, 3000, i => counter.Hit(i % 3 == 0 ? "/api/tasks" : "/health"));
Console.WriteLine($"Bài 3: {counter}");

public class Account(int id, decimal balance)
{
    private readonly Lock _gate = new();
    public int Id { get; } = id;
    public decimal Balance { get; private set; } = balance;

    public static void Transfer(Account from, Account to, decimal amount)
    {
        var (first, second) = from.Id < to.Id ? (from, to) : (to, from); // luôn lock
theo cùng thứ tự
        lock (first._gate)
        lock (second._gate)
```

```

        {
            if (from.Balance < amount) return;
            from.Balance -= amount;
            to.Balance += amount;
        }
    }
}

public class RequestCounter
{
    private readonly System.Collections.Concurrent.ConcurrentDictionary<string, long>
_counts = new();
    public void Hit(string path) => _counts.AddOrUpdate(path, 1, (_, c) => c + 1);
    public override string ToString() => string.Join(", ", _counts.OrderBy(kv =>
kv.Key).Select(kv => $"{kv.Key}={kv.Value}"));
}

```

Bài 1: a=1000, b=1000, tổng=2000 (luôn 2000)
 Bài 2: AN -> BÌNH -> CHI -> DŨNG
 Bài 3: /api/tasks=1000, /health=2000

Bài 1: khóa theo thứ tự Id tăng dần nên hai chiều chuyển tiền không bao giờ chờ vòng tròn.

Nguồn tham khảo (Sources)

- Managed threading best practices: <https://github.com/dotnet/docs/blob/main/docs/standard/threading/managed-threading-best-practices.md>
- Overview of synchronization primitives: <https://github.com/dotnet/docs/blob/main/docs/standard/threading/overview-of-synchronization-primitives.md>
- The lock statement (System.Threading.Lock): <https://github.com/dotnet/docs/blob/main/docs/csharp/language-reference/statements/lock.md>
- Data parallelism (TPL): <https://github.com/dotnet/docs/blob/main/docs/standard/parallel-programming/data-parallelism-task-parallel-library.md>
- Introduction to PLINQ: <https://github.com/dotnet/docs/blob/main/docs/standard/parallel-programming/introduction-to-plinq.md>
- Channels: <https://github.com/dotnet/docs/blob/main/docs/core/extensions/channels.md>
- SemaphoreSlim: <https://github.com/dotnet/docs/blob/main/docs/standard/threading/semaphore-and-semaphoreslim.md>

Chương 5 — Clean architecture

Mục tiêu

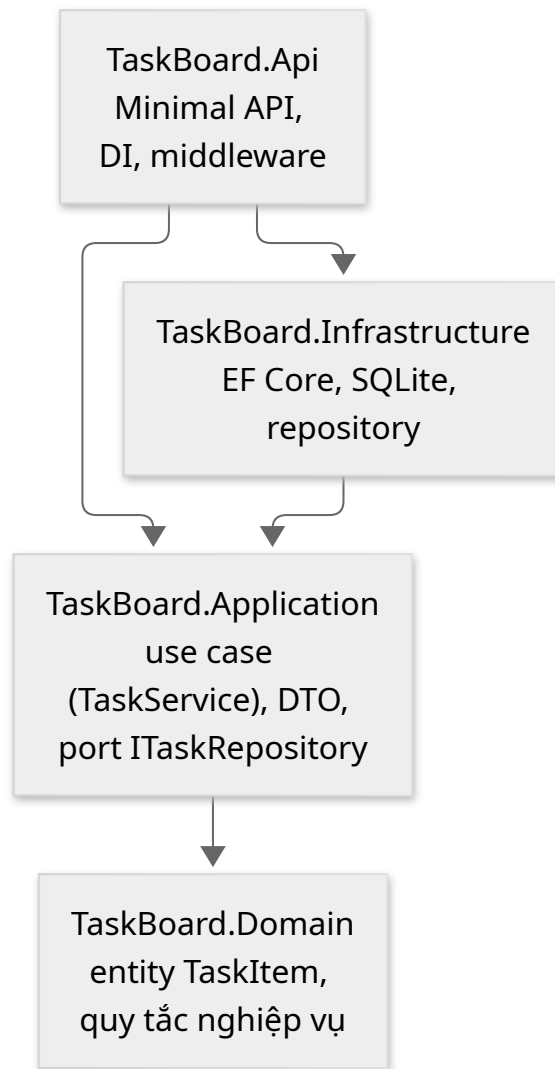
- Hiểu *clean architecture* (kiến trúc sạch) và *dependency rule* (quy tắc phụ thuộc).
- Chia một Web API thành 4 project: Domain, Application, Infrastructure, Api.
- Biết mỗi loại code nên nằm ở tầng nào.
- Kiểm tra quy tắc kiến trúc bằng test tự động.

Giải thích đơn giản

Hãy nghĩ tới một ứng dụng Angular lớn: bạn tách *component* (giao diện), *service* (logic), *HTTP client* (gọi API). Nếu logic nằm hết trong component, rất khó test và khó đổi.

Clean architecture áp dụng ý tưởng đó cho backend, với một quy tắc duy nhất:

Phụ thuộc chỉ hướng vào trong. Tầng trong (nghiệp vụ) không biết gì về tầng ngoài (database, web, framework).



- **Domain:** entity và quy tắc nghiệp vụ thuần C#. Không tham chiếu EF Core, ASP.NET Core.
- **Application:** *use case* (trường hợp sử dụng) — điều phối domain; định nghĩa **interface** (port) như `ITaskRepository` mà nó cần.
- **Infrastructure:** cài đặt interface đó bằng công nghệ cụ thể (EF Core + SQLite).
- **Api:** nhận HTTP, gọi use case, trả kết quả. Là nơi "lắp ráp" mọi thứ bằng DI.

Nhờ *dependency inversion* (đảo ngược phụ thuộc), Application gọi `ITaskRepository` mà không biết bên dưới là SQLite, PostgreSQL hay một list trong RAM (như trong unit test).

Ví dụ — dự án cuối TaskBoard

Cấu trúc thư mục `vol3-advanced/final-project/`:

```

final-project/
├─ TaskBoard.slnx
├─ global.json, Directory.Build.props, dotnet-tools.json
├─ Dockerfile, compose.yaml, .dockerignore, requests.http
├─ src/
│   ├─ TaskBoard.Domain/      TaskItem.cs (entity + rule)
│   ├─ TaskBoard.Application/  Contracts.cs (DTO, port), TaskService.cs (use cases)
│   ├─ TaskBoard.Infrastructure/ TaskBoardDbContext.cs, EfTaskRepository.cs, Migrations/
│   └─ TaskBoard.Api/         Program.cs, TaskEndpoints.cs, ApiExceptionHandler.cs
└─ tests/

```

└─ TaskBoard.UnitTests/	domain, use case, architecture tests (không DB)
└─ TaskBoard.IntegrationTests/	gọi HTTP thật vào API + SQLite in-memory

Domain: quy tắc nghiệp vụ nằm trong entity

```
namespace TaskBoard.Domain;

public enum TaskStatus
{
    Todo = 0,
    InProgress = 1,
    Done = 2,
}

public enum Priority
{
    Low = 0,
    Medium = 1,
    High = 2,
}

/// <summary>Entity: a task on the board. Business rules live here, not in the API.</summary>
public class TaskItem
{
    public const int TitleMaxLength = 200;

    // For EF Core
    private TaskItem() { Title = ""; }

    public TaskItem(string title, string? description, Priority priority, DateOnly? dueDate,
DateTimeOffset createdAt)
    {
        Title = ValidateTitle(title);
        Description = description?.Trim();
        Priority = priority;
        DueDate = dueDate;
        CreatedAt = createdAt;
        Status = TaskStatus.Todo;
    }

    public int Id { get; private set; }
    public string Title { get; private set; }
    public string? Description { get; private set; }
    public TaskStatus Status { get; private set; }
    public Priority Priority { get; private set; }
    public DateOnly? DueDate { get; private set; }
    public DateTimeOffset CreatedAt { get; private set; }
    public DateTimeOffset? CompletedAt { get; private set; }

    public void UpdateDetails(string title, string? description, Priority priority, DateOnly?
dueDate)
    {
        if (Status == TaskStatus.Done)
            throw new DomainException("Không sửa được công việc đã hoàn thành.");
        Title = ValidateTitle(title);
        Description = description?.Trim();
        Priority = priority;
        DueDate = dueDate;
    }

    /// <summary>Allowed moves: Todo → InProgress → Done, and InProgress → Todo. Done is
final.</summary>
    public void MoveTo(TaskStatus next, DateTimeOffset now)
    {
        var allowed = (Status, next) switch
        {

```

```

        (TaskStatus.TODO, TaskStatus.InProgress) => true,
        (TaskStatus.InProgress, TaskStatus.Done) => true,
        (TaskStatus.InProgress, TaskStatus.TODO) => true,
        _ => false,
    };
    if (!allowed)
        throw new DomainException($"Không chuyển được trạng thái từ {Status} sang
{next}.");
    Status = next;
    CompletedAt = next == TaskStatus.Done ? now : null;
}

public bool IsOverdue(DateOnly today) => Status != TaskStatus.Done && DueDate is { } due
&& due < today;

private static string ValidateTitle(string title)
{
    if (string.IsNullOrEmpty(title))
        throw new DomainException("Tiêu đề không được để trống.");
    title = title.Trim();
    if (title.Length > TitleMaxLength)
        throw new DomainException($"Tiêu đề tối đa {TitleMaxLength} ký tự.");
    return title;
}

public class DomainException(string message) : Exception(message);

```

Quy tắc "Done là trạng thái cuối", "không sửa công việc đã xong" nằm ở đây, không nằm trong endpoint. Mọi nơi dùng `TaskItem` đều phải tuân theo.

Application: use case và port

```

using TaskBoard.Domain;
using TaskStatus = TaskBoard.Domain.TaskStatus;

namespace TaskBoard.Application;

// DTOs: what the API receives and returns. Records = immutable, value equality.
public record CreateTaskRequest(string Title, string? Description, Priority Priority =
Priority.Medium, DateOnly? DueDate = null);

public record UpdateTaskRequest(string Title, string? Description, Priority Priority,
DateOnly? DueDate);

public record ChangeStatusRequest(TaskStatus Status);

public record TaskDto(
    int Id,
    string Title,
    string? Description,
    TaskStatus Status,
    Priority Priority,
    DateOnly? DueDate,
    bool IsOverdue,
    DateTimeOffset CreatedAt,
    DateTimeOffset? CompletedAt);

public record TaskQuery(TaskStatus? Status = null, string? Search = null, int Page = 1, int
PageSize = 20);

public record PagedResult<T>(IReadOnlyList<T> Items, int Page, int PageSize, int TotalCount);

/// <summary>Port: implemented by Infrastructure (EF Core). Application does not know about
EF.</summary>

```

```

public interface ITaskRepository
{
    Task<TaskItem?> FindAsync(int id, CancellationToken ct);
    Task<(IEnumerable<TaskItem> Items, int Total)> ListAsync(TaskQuery query,
CancellationToken ct);
    Task AddAsync(TaskItem task, CancellationToken ct);
    void Remove(TaskItem task);
    Task SaveChangesAsync(CancellationToken ct);
}

public class NotFoundException(string message) : Exception(message);

public class ValidationException(IDictionary<string, string[]> errors) : Exception("Dữ liệu
không hợp lệ.")
{
    public IDictionary<string, string[]> Errors { get; } = errors;
}

```

```

using Microsoft.Extensions.Logging;
using TaskBoard.Domain;

namespace TaskBoard.Application;

/// <summary>Use cases of the application. Depends only on abstractions (ports).</summary>
public class TaskService(ITaskRepository repository, TimeProvider clock, ILogger<TaskService>
logger)
{
    private DateOnly Today => DateOnly.FromDateTime(clock.GetUtcNow().UtcDateTime);

    public async Task<TaskDto> CreateAsync(CreateTaskRequest request, CancellationToken ct =
default)
    {
        Validate(request.Title, request.DueDate, isNew: true);
        var task = new TaskItem(request.Title, request.Description, request.Priority,
request.DueDate, clock.GetUtcNow());
        await repository.AddAsync(task, ct);
        await repository.SaveChangesAsync(ct);
        logger.LogInformation("Created task {TaskId} with priority {Priority}", task.Id,
task.Priority);
        return ToDto(task);
    }

    public async Task<TaskDto> GetAsync(int id, CancellationToken ct = default)
        => ToDto(await FindOrThrowAsync(id, ct));

    public async Task<PagedResult<TaskDto>> ListAsync(TaskQuery query, CancellationToken ct =
default)
    {
        var errors = new Dictionary<string, string[]>();
        if (query.Page < 1) errors["page"] = ["page phải >= 1."];
        if (query.PageSize is < 1 or > 100) errors["pageSize"] = ["pageSize phải từ 1 đến
100."];
        if (errors.Count > 0) throw new ValidationException(errors);

        var (items, total) = await repository.ListAsync(query, ct);
        return new PagedResult<TaskDto>(items.Select(ToDto).ToList(), query.Page,
query.PageSize, total);
    }

    public async Task<TaskDto> UpdateAsync(int id, UpdateTaskRequest request,
CancellationToken ct = default)
    {
        Validate(request.Title, request.DueDate, isNew: false);
        var task = await FindOrThrowAsync(id, ct);
        task.UpdateDetails(request.Title, request.Description, request.Priority,
request.DueDate);
        await repository.SaveChangesAsync(ct);
    }
}

```

```

        return ToDto(task);
    }

    public async Task<TaskDto> ChangeStatusAsync(int id, ChangeStatusRequest request,
CancellationTokens ct = default)
    {
        var task = await FindOrThrowAsync(id, ct);
        var old = task.Status;
        task.MoveTo(request.Status, clock.UtcNow());
        await repository.SaveChangesAsync(ct);
        logger.LogInformation("Task {TaskId} moved {OldStatus} -> {NewStatus}", id, old,
task.Status);
        return ToDto(task);
    }

    public async Task DeleteAsync(int id, CancellationTokens ct = default)
    {
        var task = await FindOrThrowAsync(id, ct);
        repository.Remove(task);
        await repository.SaveChangesAsync(ct);
        logger.LogInformation("Deleted task {TaskId}", id);
    }

    private async Task<TaskItem> FindOrThrowAsync(int id, CancellationTokens ct)
    => await repository.FindAsync(id, ct) ?? throw new NotFoundException($"Không tìm thấy
công việc {id}.");

    private void Validate(string? title, DateOnly? dueDate, bool isNew)
    {
        var errors = new Dictionary<string, string[]>();
        if (string.IsNullOrEmpty(title))
            errors["title"] = ["Tiêu đề là bắt buộc."];
        else if (title.Trim().Length > TaskItem.TitleMaxLength)
            errors["title"] = [$"Tiêu đề tối đa {TaskItem.TitleMaxLength} ký tự."];
        if (isNew && dueDate is { } due && due < Today)
            errors["dueDate"] = ["Hạn chót không được ở quá khứ."];
        if (errors.Count > 0) throw new ValidationException(errors);
    }

    private TaskDto ToDto(TaskItem t) => new(
        t.Id, t.Title, t.Description, t.Status, t.Priority, t.DueDate, t.IsOverdue(Today),
t.CreatedAt, t.CompletedAt);
    }

```

TaskService nhận TimeProvider thay vì gọi DateTime.UtcNow → test điều khiển được thời gian.

Infrastructure: cài đặt port bằng EF Core

```

using Microsoft.EntityFrameworkCore;
using TaskBoard.Application;
using TaskBoard.Domain;

namespace TaskBoard.Infrastructure;

public class EfTaskRepository(TaskBoardDbContext db) : ITaskRepository
{
    public Task<TaskItem?> FindAsync(int id, CancellationTokens ct) =>
        db.Tasks.FirstOrDefaultAsync(t => t.Id == id, ct);

    public async Task<(IReadOnlyList<TaskItem> Items, int Total)> ListAsync(TaskQuery query,
CancellationTokens ct)
    {
        IQueryable<TaskItem> q = db.Tasks.AsNoTracking();
        if (query.Status is { } status) q = q.Where(t => t.Status == status);
        if (!string.IsNullOrEmpty(query.Search))

```

```

        q = q.Where(t => EF.Functions.Like(t.Title, $"%{query.Search.Trim()}%"));

        int total = await q.CountAsync(ct);
        var items = await q
            .OrderBy(t => t.Status == Domain.TaskStatus.Done)    // unfinished first
            .ThenByDescending(t => t.Priority)
            .ThenBy(t => t.Id)
            .Skip((query.Page - 1) * query.PageSize)
            .Take(query.PageSize)
            .ToListAsync(ct);
        return (items, total);
    }

    public async Task AddAsync(TaskItem task, CancellationToken ct) => await
db.Tasks.AddAsync(task, ct);

    public void Remove(TaskItem task) => db.Tasks.Remove(task);

    public Task SaveChangesAsync(CancellationToken ct) => db.SaveChangesAsync(ct);
}

```

```

using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using TaskBoard.Application;

namespace TaskBoard.Infrastructure;

public static class DependencyInjection
{
    public const string ConnectionStringName = "TaskBoard";

    public static IServiceCollection AddInfrastructure(this IServiceCollection services)
    {
        // The connection string is read when the DbContext is created (not at startup),
        // so tests can override it through configuration.
        services.AddDbContext<TaskBoardDbContext>((sp, options) =>
            options.UseSqlite(sp.GetRequiredService<IConfiguration>
                ().GetConnectionString(ConnectionStringName)
                    ?? "Data Source=taskboard.db"));
        services.AddScoped<ITaskRepository, EfTaskRepository>();
        return services;
    }
}

```

Api: endpoint mỏng, lỗi xử lý tập trung

```

using System.Text.Json.Serialization;
using Microsoft.EntityFrameworkCore;
using TaskBoard.Api;
using TaskBoard.Application;
using TaskBoard.Infrastructure;

var builder = WebApplication.CreateBuilder(args);

// ---- Services (DI) ----
builder.Services.AddInfrastructure();    // EF Core + SQLite (connection string "TaskBoard")
builder.Services.AddSingleton(TimeProvider.System);
builder.Services.AddScoped<TaskService>();

builder.Services.ConfigureHttpJsonOptions(o =>
    o.SerializerOptions.Converters.Add(new JsonStringEnumConverter()));
builder.Services.AddProblemDetails();
builder.Services.AddExceptionHandler<ApiExceptionHandler>();
builder.Services.AddOpenApi();

```

```

builder.Services.AddHealthChecks().AddCheck<DatabaseHealthCheck>("database");

var app = builder.Build();

// ---- Database migration on startup (simple apps / demos) ----
if (app.Configuration.GetValue("Database:MigrateOnStartup", true))
{
    using var scope = app.Services.CreateScope();
    scope.ServiceProvider.GetRequiredService<TaskBoardDbContext>().Database.Migrate();
}

// ---- Middleware pipeline ----
app.UseExceptionHandler();
app.UseStatusCodePages();

app.MapOpenApi(); // GET /openapi/v1.json
app.MapHealthChecks("/health"); // GET /health
app.MapGet("/", () => Results.Redirect("/openapi/v1.json")).ExcludeFromDescription();
app.MapTaskEndpoints(); // /api/tasks

app.Run();

// Makes Program visible to WebApplicationFactory<Program> in the integration tests.
public partial class Program;

```

```

using Microsoft.AspNetCore.Http.HttpResults;
using TaskBoard.Application;
using TaskStatus = TaskBoard.Domain.TaskStatus;

namespace TaskBoard.Api;

public static class TaskEndpoints
{
    public static IEndpointRouteBuilder MapTaskEndpoints(this IEndpointRouteBuilder app)
    {
        var group = app.MapGroup("/api/tasks").WithTags("Tasks");

        group.MapGet("/", async (TaskService service, TaskStatus? status, string? search,
            int page = 1, int pageSize = 20, CancellationToken ct = default) =>
            TypedResults.Ok(await service.ListAsync(new TaskQuery(status, search, page,
                pageSize), ct)))
            .WithName("ListTasks");

        group.MapGet("/{id:int}", async (int id, TaskService service, CancellationToken ct) =>
            TypedResults.Ok(await service.GetAsync(id, ct)))
            .WithName("GetTask");

        group.MapPost("/", async Task<Created<TaskDto>> (CreateTaskRequest request,
            TaskService service, CancellationToken ct) =>
            {
                var created = await service.CreateAsync(request, ct);
                return TypedResults.Created($"/api/tasks/{created.Id}", created);
            }).WithName("CreateTask");

        group.MapPut("/{id:int}", async (int id, UpdateTaskRequest request, TaskService
            service, CancellationToken ct) =>
            TypedResults.Ok(await service.UpdateAsync(id, request, ct)))
            .WithName("UpdateTask");

        group.MapPatch("/{id:int}/status", async (int id, ChangeStatusRequest request,
            TaskService service, CancellationToken ct) =>
            TypedResults.Ok(await service.ChangeStatusAsync(id, request, ct)))
            .WithName("ChangeTaskStatus");

        group.MapDelete("/{id:int}", async Task<NoContent> (int id, TaskService service,
            CancellationToken ct) =>
            {

```

```

        await service.DeleteAsync(id, ct);
        return TypedResults.NoContent();
    }).WithName("DeleteTask");

    return app;
}
}

```

```

using Microsoft.AspNetCore.Diagnostics;
using Microsoft.AspNetCore.Mvc;
using TaskBoard.Application;
using TaskBoard.Domain;

namespace TaskBoard.Api;

/// <summary>Maps exceptions to Problem Details (RFC 7807 / 9457) responses in one place.
</summary>
public sealed class ApiExceptionHandler(IProblemDetailsService problemDetails,
    ILogger<ApiExceptionHandler> logger)
    : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext http, Exception exception,
        CancellationToken ct)
    {
        ProblemDetails problem = exception switch
        {
            ValidationException v => new ValidationProblemDetails(v.Errors)
            {
                Status = StatusCodes.Status400BadRequest,
                Title = v.Message,
            },
            NotFoundException nf => new ProblemDetails { Status =
                StatusCodes.Status404NotFound, Title = nf.Message },
            DomainException d => new ProblemDetails { Status = StatusCodes.Status409Conflict,
                Title = d.Message },
            BadRequestException b => new ProblemDetails { Status =
                StatusCodes.Status400BadRequest, Title = b.Message },
            _ => new ProblemDetails { Status = StatusCodes.Status500InternalServerError, Title
                = "Lỗi hệ thống." },
        };

        if (problem.Status == StatusCodes.Status500InternalServerError)
            logger.LogError(exception, "Unhandled exception for {Method} {Path}",
                http.Request.Method, http.Request.Path);
        else
            logger.LogInformation("Request {Method} {Path} failed with {Status}: {Title}",
                http.Request.Method, http.Request.Path, problem.Status, problem.Title);

        http.Response.StatusCode = problem.Status!.Value;
        return await problemDetails.TryWriteAsync(new ProblemDetailsContext
        {
            HttpContext = http,
            ProblemDetails = problem,
            Exception = exception,
        });
    }
}

```

Endpoint chỉ có một dòng: gọi use case. Exception nghiệp vụ được đổi sang HTTP status ở **một chỗ** (ApiExceptionHandler): ValidationException → 400, NotFoundException → 404, DomainException → 409.

Kiểm tra quy tắc kiến trúc bằng test

```
using TaskBoard.Application;
using TaskBoard.Domain;

namespace TaskBoard.UnitTests;

/// <summary>Checks the dependency rule of clean architecture: inner layers do not know outer
layers.</summary>
public class ArchitectureTests
{
    private static string[] ReferencedNames(Type typeInAssembly) =>
        typeInAssembly.Assembly.GetReferencedAssemblies().Select(a => a.Name!).ToArray();

    [Fact]
    public void Domain_DependsOnNoOtherLayerOrFramework()
    {
        var refs = ReferencedNames(typeof(TaskItem));
        Assert.DoesNotContain(refs, n => n.StartsWith("TaskBoard.",
StringComparison.Ordinal));
        Assert.DoesNotContain(refs, n => n.StartsWith("Microsoft.EntityFrameworkCore",
StringComparison.Ordinal));
        Assert.DoesNotContain(refs, n => n.StartsWith("Microsoft.AspNetCore",
StringComparison.Ordinal));
    }

    [Fact]
    public void Application_DependsOnDomainOnly_NotOnInfrastructureOrEfCore()
    {
        var refs = ReferencedNames(typeof(TaskService));
        Assert.Contains("TaskBoard.Domain", refs);
        Assert.DoesNotContain("TaskBoard.Infrastructure", refs);
        Assert.DoesNotContain("TaskBoard.Api", refs);
        Assert.DoesNotContain(refs, n => n.StartsWith("Microsoft.EntityFrameworkCore",
StringComparison.Ordinal));
    }
}
```

Kết quả chạy toàn bộ test của dự án cuối (unit + integration + architecture), output thật:

```
$ dotnet build TaskBoard.slnx
  0 Warning(s)
  0 Error(s)

$ dotnet test --solution TaskBoard.slnx --output Detailed
Running tests from <repo>/books/csharp/vol3-advanced/final-
project/tests/TaskBoard.UnitTests/bin/Debug/net10.0/TaskBoard.UnitTests.dll (net10.0|x64)
Running tests from <repo>/books/csharp/vol3-advanced/final-
project/tests/TaskBoard.IntegrationTests/bin/Debug/net10.0/TaskBoard.IntegrationTests.dll
(net10.0|x64)
passed TaskBoard.UnitTests.TaskItemTests.NewTask_StartsInTodo_WithTrimmedTitle (24ms)
passed TaskBoard.UnitTests.TaskServiceTests.Create_InvalidInput_ThrowsValidationWithAllErrors (27ms)
passed
TaskBoard.UnitTests.ArchitectureTests.Application_DependsOnDomainOnly_NotOnInfrastructureOrEfCore
(25ms)
passed TaskBoard.UnitTests.ArchitectureTests.Domain_DependsOnNoOtherLayerOrFramework (0ms)
passed TaskBoard.UnitTests.TaskItemTests.DoneTask_CannotBeEdited (2ms)
passed TaskBoard.UnitTests.TaskItemTests.MoveTo_FollowsWorkflow_AndSetsCompletedAt (4ms)
passed TaskBoard.UnitTests.TaskServiceTests.Create_ReturnsDto_AndSaves (16ms)
passed TaskBoard.UnitTests.TaskServiceTests.List_InvalidPaging_Throws(page: 1, pageSize: 101) (1ms)
passed TaskBoard.UnitTests.TaskServiceTests.List_InvalidPaging_Throws(page: 0, pageSize: 20) (0ms)
passed TaskBoard.UnitTests.TaskServiceTests.List_InvalidPaging_Throws(page: 1, pageSize: 0) (0ms)
passed TaskBoard.UnitTests.TaskServiceTests.Overdue_IsComputedFromClock (2ms)
passed TaskBoard.UnitTests.TaskServiceTests.Get_Missing_ThrowsNotFound (0ms)
passed TaskBoard.UnitTests.TaskItemTests.MoveTo_InvalidTransition_Throws(from: Todo, to: Done) (15ms)
passed TaskBoard.UnitTests.TaskServiceTests.ChangeStatus_ToDone_SetsCompletedAtFromClock (2ms)
```

```

passed TaskBoard.UnitTests.TaskItemTests.MoveTo_InvalidTransition_Throws(from: Todo, to: Todo) (0ms)
passed TaskBoard.UnitTests.TaskItemTests.IsOverdue_OnlyWhenNotDoneAndPastDue (0ms)
passed TaskBoard.UnitTests.TaskItemTests.NewTask_BlankTitle_Throws(title: " ") (0ms)
passed TaskBoard.UnitTests.TaskItemTests.NewTask_BlankTitle_Throws(title: "") (0ms)
passed TaskBoard.UnitTests.TaskServiceTests.List_FiltersBySearch (3ms)
<repo>/books/csharp/vol3-advanced/final-
project/tests/TaskBoard.UnitTests/bin/Debug/net10.0/TaskBoard.UnitTests.dll (net10.0|x64) passed
(858ms)
passed TaskBoard.IntegrationTests.TasksApiTests.Post_ThenGet_ReturnsSameTask_WithLocationHeader (1s
553ms)
passed TaskBoard.IntegrationTests.TasksApiTests.List_FiltersByStatusAndSearch_WithPaging (90ms)
passed TaskBoard.IntegrationTests.TasksApiTests.Health_ReturnsHealthy (16ms)
passed TaskBoard.IntegrationTests.TasksApiTests.Put_UpdatesTask (16ms)
passed TaskBoard.IntegrationTests.TasksApiTests.OpenApi_DocumentIsServed (161ms)
passed TaskBoard.IntegrationTests.TasksApiTests.Status_Workflow_AndInvalidTransition_Returns409 (37ms)
passed TaskBoard.IntegrationTests.TasksApiTests.Delete_RemovesTask (14ms)
passed TaskBoard.IntegrationTests.TasksApiTests.Get_Unknown_Returns404 (2ms)
passed TaskBoard.IntegrationTests.TasksApiTests.Post_InvalidBody_Returns400ProblemDetails (7ms)
<repo>/books/csharp/vol3-advanced/final-
project/tests/TaskBoard.IntegrationTests/bin/Debug/net10.0/TaskBoard.IntegrationTests.dll
(net10.0|x64) passed (2s 716ms)

Test run summary: Passed!
<repo>/books/csharp/vol3-advanced/final-
project/tests/TaskBoard.UnitTests/bin/Debug/net10.0/TaskBoard.UnitTests.dll (net10.0|x64) passed
(858ms)
<repo>/books/csharp/vol3-advanced/final-
project/tests/TaskBoard.IntegrationTests/bin/Debug/net10.0/TaskBoard.IntegrationTests.dll
(net10.0|x64) passed (2s 716ms)

total: 28
failed: 0
succeeded: 28
skipped: 0
duration: 3s 000ms

```

Đi sâu

Code nào nằm ở đâu?

Loại code	Tầng
Entity, value object, quy tắc nghiệp vụ, <code>DomainException</code>	Domain
Use case, DTO request/response, interface repository/email/clock	Application
<code>DbContext</code> , migration, repository EF Core, gửi email SMTP, gọi API ngoài	Infrastructure
Endpoint, middleware, cấu hình DI, OpenAPI, health check	Api

Test theo tầng

- **Unit test** (nhANH, không DB): Domain và Application với fake repository, `FixedTimeProvider`.
- **Integration test**: `WebApplicationFactory<Program>` chạy API thật trong bộ nhớ, SQLite in-memory, gọi HTTP thật → kiểm tra routing, JSON, status code, EF Core mapping.

Khi nào không cần 4 project?

Clean architecture có chi phí: nhiều project, nhiều file, nhiều mapping. Với API nhỏ (vài endpoint, một người làm), một project chia thư mục (`Domain/`, `Features/` ...) là đủ. Hãy bắt đầu đơn giản và tách khi code lớn lên. Một lựa chọn khác phổ biến là *vertical slice architecture* (chia theo tính năng

thay vì theo tầng).

Template tham khảo (reputable repos, checked 2026-09-28)

URL	Stars	Last commit	License	Hữu ích	Checked
https://github.com/jasontaylordev/CleanArchitecture	~20.6k	2026-09-28	MIT	template dotnet new, .NET 10, có frontend Angular/React	2026-09-28
https://github.com/ardalis/CleanArchitecture	~18.5k	2026-09-21	MIT	template ASP.NET Core, nhiều ghi chú giải thích	2026-09-28

Số sao và ngày commit đọc từ trang GitHub bằng công cụ fetch ngày 2026-09-28; có thể thay đổi.

Lỗi và bẫy thường gặp

- **Domain tham chiếu EF Core** (attribute `[Key]`, `DbSet`) → vi phạm dependency rule. Dùng Fluent API trong Infrastructure (`OnModelCreating`).
- **Logic nghiệp vụ trong endpoint/controller** → trùng lặp, khó test.
- **Entity có setter public khắp nơi** → ai cũng sửa được, bỏ qua quy tắc. Dùng `private set` và `method`.
- **Repository "chung chung" bọc mọi thứ của EF Core** (`IRepository<T>` với 30 method) → thêm lớp mà không thêm giá trị. Chỉ định nghĩa method use case cần.
- **Trả entity ra API** thay vì DTO.
- **Quá nhiều tầng cho project nhỏ** (xem mục "Khi nào không cần").

Tóm tắt

- Dependency rule: phụ thuộc hướng vào trong; Domain không biết gì về framework.
- Application định nghĩa port; Infrastructure cài đặt; Api lắp ráp bằng DI.
- Test kiến trúc giúp quy tắc không bị phá vỡ theo thời gian.

Bài tập (có lời giải)

1. Với mỗi đoạn code sau, nó thuộc tầng nào? (a) quy tắc "hạn chót không ở quá khứ khi tạo mới"; (b) `CREATE INDEX` cho cột `Status`; (c) chuyển `NotFoundException` thành `HTTP 404`; (d) interface `ISender`; (e) class `SmtEmailSender`.
2. Vì sao `TaskService` nhận `TimeProvider` thay vì dùng `DateTime.UtcNow`?
3. Thêm một test kiến trúc: Domain không được tham chiếu `Microsoft.AspNetCore`.

▼ Lời giải

1. (a) Application — đây là kiểm tra dữ liệu đầu vào của use case "tạo mới" (xem `Validate` trong `TaskService`); quy tắc bất biến của entity (tiêu đề không rỗng) nằm ở Domain. (b) Infrastructure (`HasIndex` trong `TaskBoardDbContext`, sinh ra migration). (c) Api (`ApiExceptionHandler`). (d) Application (port). (e) Infrastructure (adapter).

2. Để test điều khiển được thời gian: `FixedTimeProvider` trong unit test đặt "bây giờ" là 2026-09-28 rồi tua tới 2026-10-02 để kiểm tra `IsOverdue` (test `Overdue_IsComputedFromClock`).
3. Test đã có trong `ArchitectureTests.Domain_DependsOnNoOtherLayerOrFramework` (dòng kiểm tra `Microsoft.AspNetCore`), và pass trong output ở trên.

Nguồn tham khảo (Sources)

- Common web application architectures (nguồn Microsoft Learn, e-book .NET): <https://github.com/dotnet/docs/blob/main/docs/architecture/modern-web-apps-azure/common-web-application-architectures.md>
- Architectural principles (dependency inversion, separation of concerns): <https://github.com/dotnet/docs/blob/main/docs/architecture/modern-web-apps-azure/architectural-principles.md>
- Integration tests in ASP.NET Core: <https://github.com/dotnet/AspNetCore.Docs/blob/main/aspnetcore/test/integration-tests.md>
- TimeProvider (.NET 8+): <https://github.com/dotnet/docs/blob/main/docs/standard/datetime/time-provider-overview.md>
- jasontaylordev/CleanArchitecture: <https://github.com/jasontaylordev/CleanArchitecture>
- ardalis/CleanArchitecture: <https://github.com/ardalis/CleanArchitecture>

Chương 6 — Logging và observability

Mục tiêu

- Dùng `ILogger<T>` với log level, filter, *structured logging* (log có cấu trúc) và scope.
- Viết log hiệu năng cao bằng `[LoggerMessage]` (source generator).
- Hiểu 3 trụ cột *observability* (khả năng quan sát): logs, traces, metrics.
- Thêm traces/metrics bằng `ActivitySource`, `Meter` và `OpenTelemetry`.
- Dùng health check cho `live` / `ready`.

Giải thích đơn giản

Khi API chạy trên server, bạn không thể đặt breakpoint. Bạn cần **dấu vết**:

Trụ cột	Trả lời câu hỏi	.NET API
Logs	Chuyện gì đã xảy ra?	<code>ILogger</code>
Traces	Một request đi qua những bước nào, mỗi bước bao lâu?	<code>System.Diagnostics.Activity</code> / <code>ActivitySource</code>
Metrics	Bao nhiêu? Nhanh hay chậm? (đếm, đo theo thời gian)	<code>System.Diagnostics.Metrics.Meter</code>

OpenTelemetry (OTel) là chuẩn mở để thu thập và gửi 3 loại dữ liệu trên tới công cụ như Jaeger, Prometheus, Grafana, Azure Monitor... .NET có sẵn API; OTel SDK chỉ việc "xuất" dữ liệu đi.

So với frontend: giống `console.log` + Sentry + Google Analytics, nhưng cho backend và theo chuẩn chung.

Ví dụ

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.Hosting" Version="10.0.12" />
    <PackageReference Include="OpenTelemetry.Extensions.Hosting" Version="1.19.1" />
    <PackageReference Include="OpenTelemetry.Exporter.Console" Version="1.19.1" />
  </ItemGroup>

</Project>
```

```
using System.Diagnostics;
using System.Diagnostics.Metrics;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using OpenTelemetry.Metrics;
using OpenTelemetry.Resources;
using OpenTelemetry.Trace;
```

```

var builder = Host.CreateApplicationBuilder(args);

// 1. Logging: JSON console formatter -> mỗi dòng log là một JSON có các trường riêng
builder.Logging.ClearProviders();
builder.Logging.AddJsonConsole(o =>
{
    o.IncludeScopes = true;
    o.JsonWriterOptions = new System.Text.Json.JsonWriterOptions
    {
        Indented = false,
        Encoder = System.Text.Encodings.Web.JavaScriptEncoder.UnsafeRelaxedJsonEscaping, //
        giữ nguyên tiếng Việt
    };
});
builder.Logging.SetMinimumLevel(LogLevel.Information);
builder.Logging.AddFilter("Microsoft", LogLevel.Warning); // bắt log của framework

// 2. Traces + metrics với OpenTelemetry, xuất ra console (thực tế: OTLP ->
// Jaeger/Grafana/...)
builder.Services.AddOpenTelemetry()
    .ConfigureResource(r => r.AddService("checkout-demo"))
    .WithTracing(t => t.AddSource(Checkout.ActivitySourceName).AddConsoleExporter())
    .WithMetrics(m => m.AddMeter(Checkout.MeterName).AddConsoleExporter(_, reader) =>
        reader.PeriodicExportingMetricReaderOptions.ExportIntervalMilliseconds = 60_000));

builder.Services.AddSingleton<Checkout>();
using var host = builder.Build();
await host.StartAsync();

var checkout = host.Services.GetRequiredService<Checkout>();
checkout.PlaceOrder("ORD-1001", 3, 450_000m);
checkout.PlaceOrder("ORD-1002", 0, 0m);

await host.StopAsync(); // flush traces và metrics ra console

public partial class Checkout(ILogger<Checkout> logger)
{
    public const string ActivitySourceName = "Demo.Checkout";
    public const string MeterName = "Demo.Checkout";
    private static readonly ActivitySource Source = new(ActivitySourceName);
    private static readonly Meter Meter = new(MeterName);
    private static readonly Counter<long> OrdersPlaced = Meter.CreateCounter<long>
("orders.placed");

    public void PlaceOrder(string orderId, int quantity, decimal total)
    {
        using var activity = Source.StartActivity("PlaceOrder"); // một "span" trong trace
        activity?.SetTag("order.id", orderId);

        // Scope: mọi log bên trong đều mang OrderId
        using var scope = logger.BeginScope("Order {OrderId}", orderId);

        if (quantity <= 0)
        {
            LogInvalidQuantity(quantity);
            activity?.SetStatus(ActivityStatusCode.Error, "invalid quantity");
            return;
        }

        // Structured logging: {Quantity}, {Total} là tên trường, KHÔNG dùng $"..."
        logger.LogInformation("Đặt hàng thành công: {Quantity} sản phẩm, tổng {Total}",
quantity, total);
        OrdersPlaced.Add(1, new KeyValuePair<string, object?>("status", "ok"));
    }

    // Source-generated logging: nhanh hơn, không cấp phát khi level bị tắt

```

```

        [LoggerMessage(EventId = 4001, Level = LogLevel.Warning, Message = "Số lượng không hợp lệ: {Quantity}")]
        private partial void LogInvalidQuantity(int quantity);
    }

```

Output thật (log JSON, trace và metric xuất ra console; `TraceId`/`SpanId` ngẫu nhiên mỗi lần chạy. Logger console và exporter OTel ghi từ hai luồng khác nhau nên thứ tự các dòng có thể xen nhau):

```

{"EventId":0,"LogLevel":"Information","Category":"Checkout","Message":"Đặt hàng thành công: 3 sản phẩm, tổng 450000","State":{"Quantity":3,"Total":450000,"{OriginalFormat}":"Đặt hàng thành công: {Quantity} sản phẩm, tổng {Total}"}, "Scopes":[{"Message":"SpanId:0b22fa4fea47e66d, TraceId:443fcd936bc47f4c8bd6b4524216bcfe, ParentId:0000000000000000","SpanId":"0b22fa4fea47e66d","TraceId":"443fcd936bc47f4c8bd6b4524216bcfe","ParentId":"0000000000000000"}, {"Message":"Order ORD-1001","OrderId":"ORD-1001","{OriginalFormat}":"Order {OrderId}"}]}
Activity.TraceId: 443fcd936bc47f4c8bd6b4524216bcfe
Activity.SpanId: 0b22fa4fea47e66d
Activity.TraceFlags: Recorded
Activity.DisplayName: PlaceOrder
Activity.Kind: Internal
Activity.StartTime: 2026-09-28T15:27:44.4046078Z
Activity.Duration: 00:00:00.0194751
Activity.Tags:
  order.id: ORD-1001
Instrumentation scope (ActivitySource):
  Name: Demo.Checkout
Resource associated with Activity:
  service.name: checkout-demo
  service.instance.id: ce650502-35aa-41e2-be36-92460c22b15c
  telemetry.sdk.name: opentelemetry
  telemetry.sdk.language: dotnet
  telemetry.sdk.version: 1.19.1
  Schema URL: https://opentelemetry.io/schemas/1.44.0

Activity.TraceId: 4fa0fd141a2c5c754cc57c1139e652d4
Activity.SpanId: 00fc3b00d485e8db
Activity.TraceFlags: Recorded
{"EventId":4001,"LogLevel":"Warning","Category":"Checkout","Message":"Số lượng không hợp lệ: 0","State":{"Quantity":0,"{OriginalFormat}":"Số lượng không hợp lệ: {Quantity}"}, "Scopes":[{"Message":"SpanId:00fc3b00d485e8db, TraceId:4fa0fd141a2c5c754cc57c1139e652d4, ParentId:0000000000000000","SpanId":"00fc3b00d485e8db","TraceId":"4fa0fd141a2c5c754cc57c1139e652d4","ParentId":"0000000000000000"}, {"Message":"Order ORD-1002","OrderId":"ORD-1002","{OriginalFormat}":"Order {OrderId}"}]}
Activity.DisplayName: PlaceOrder
Activity.Kind: Internal
Activity.StartTime: 2026-09-28T15:27:44.4359290Z
Activity.Duration: 00:00:00.0052853
Activity.Tags:
  order.id: ORD-1002
StatusCode: Error
Activity.StatusDescription: invalid quantity
Instrumentation scope (ActivitySource):
  Name: Demo.Checkout
Resource associated with Activity:
  service.name: checkout-demo
  service.instance.id: ce650502-35aa-41e2-be36-92460c22b15c
  telemetry.sdk.name: opentelemetry
  telemetry.sdk.language: dotnet
  telemetry.sdk.version: 1.19.1
  Schema URL: https://opentelemetry.io/schemas/1.44.0

Resource associated with Metrics:
  service.name: checkout-demo
  service.instance.id: ce650502-35aa-41e2-be36-92460c22b15c
  telemetry.sdk.name: opentelemetry
  telemetry.sdk.language: dotnet

```

```
telemetry.sdk.version: 1.19.1
Schema URL: https://opentelemetry.io/schemas/1.44.0
```

```
Metric Name: orders.placed, Metric Type: LongSum
Instrumentation scope (Meter):
  Name: Demo.Checkout
(2026-09-28T15:27:44.3962990Z, 2026-09-28T15:27:44.4549131Z] status: ok
Value: 1
```

Để ý trong log JSON:

- `State` có trường riêng `Quantity: 3, Total: 450000` → công cụ log lọc được "mọi đơn có Total > 1 triệu" mà không phải phân tích chuỗi.
- `Scopes` chứa `OrderId` và `TraceId/SpanId` → nối log với trace tương ứng.
- Log cảnh báo có `EventId: 4001` từ `[LoggerMessage]`.

Đi sâu

Log level

Level	Dùng cho
Trace, Debug	chi tiết khi phát triển; tắt ở production
Information	sự kiện nghiệp vụ bình thường (đặt hàng, đăng nhập)
Warning	bất thường nhưng vẫn chạy (dữ liệu sai, retry)
Error	thao tác thất bại (exception)
Critical	hệ thống sắp/đã ngừng

Cấu hình trong `appsettings.json` (dự án cuối):

```
"Logging": { "LogLevel": { "Default": "Information", "Microsoft.AspNetCore": "Warning" } }
```

Structured logging: đúng và sai

```
logger.LogInformation("Đặt hàng {OrderId} tổng {Total}", id, total); // ĐÚNG: có trường
OrderId, Total
logger.LogInformation($"Đặt hàng {id} tổng {total}"); // SAI: chỉ là một
chuỗi
```

Cách sai mất cấu trúc, và luôn tạo chuỗi kể cả khi level bị tắt.

[LoggerMessage]

Source generator sinh code log tối ưu: kiểm tra level trước, không boxing tham số, có `EventId` cố định. Dùng cho log được gọi nhiều (mỗi request).

Traces và correlation

ASP.NET Core tự tạo một `Activity` cho mỗi request HTTP. `TraceId` được truyền qua header `traceparent` (chuẩn W3C) sang dịch vụ khác → xem được toàn bộ hành trình của một request qua nhiều service. Problem Details của dự án cuối trả về `traceId` để người dùng báo lỗi kèm mã này.

Health checks

- **Liveness** (/health/live): process còn sống? Nếu fail → orchestrator (Kubernetes) khởi động lại.
- **Readiness** (/health/ready): sẵn sàng nhận traffic chưa (DB, dịch vụ ngoài)? Nếu fail → tạm ngưng gửi request tới.
- Dự án cuối có /health kiểm tra kết nối SQLite (DatabaseHealthCheck).

Đưa lên production

Thay `AddConsoleExporter()` bằng `AddOtlpExporter()` (gói `OpenTelemetry.Exporter.OpenTelemetryProtocol`) để gửi tới collector. Thêm `AddAspNetCoreInstrumentation()` (gói `OpenTelemetry.Instrumentation.AspNetCore`) để có trace/metric HTTP tự động. **UNVERIFIED** trong bộ sách: phần xuất OTLP tới collector thật không chạy ở đây vì không có collector.

Lỗi và bẫy thường gặp

- **Log dữ liệu nhạy cảm** (mật khẩu, token, số thẻ, thông tin cá nhân) → vi phạm bảo mật/pháp lý.
- **Dùng "\$" trong log** → mất cấu trúc, tốn CPU.
- **Log quá nhiều ở Information** trong vòng lặp nóng → tốn chi phí lưu trữ, khó tìm.
- **Nuốt exception mà không log** hoặc log mà không kèm object exception (`LogError(ex, ...)`).
- **Health check "ready" gọi dịch vụ ngoài cho cả liveness** → dịch vụ ngoài chậm làm container bị khởi động lại liên tục. Tách live/ready như Bài tập 3.
- **Tên metric/tag có giá trị không giới hạn** (user id, URL đầy đủ) → bùng nổ số chuỗi thời gian.

Tóm tắt

- `ILogger` + structured logging + scope; `[LoggerMessage]` cho đường nóng.
- Observability = logs + traces + metrics; OpenTelemetry là chuẩn xuất dữ liệu.
- Health check tách live và ready.

Bài tập (có lời giải)

1. Thêm health check tùy chỉnh kiểm tra dung lượng đĩa trống, và một check "payment-api" giả lập lỗi.
2. Viết middleware log mỗi request (method, path, status) bằng `[LoggerMessage]`, trong scope `CorrelationId` lấy từ header `X-Correlation-Id`.
3. Tách `/health/live` (bỏ qua check có tag `external`) và `/health/ready` (tất cả check).

▼ Lời giải

```
using Microsoft.Extensions.Diagnostics.HealthChecks;

var builder = WebApplication.CreateBuilder(args);
builder.WebHost.UseUrls("http://127.0.0.1:5197");
builder.Logging.ClearProviders();
```

```

builder.Logging.AddSimpleConsole(o => { o.SingleLine = true; o.IncludeScopes = true; });
builder.Logging.AddFilter("Microsoft", LogLevel.Warning);

// Bài 1: health check tùy chỉnh kiểm tra dung lượng đĩa trống (ngưỡng giả lập)
builder.Services.AddHealthChecks()
    .AddCheck("disk", () =>
    {
        long freeMb = new DriveInfo("/").AvailableFreeSpace / 1024 / 1024;
        return freeMb > 100
            ? HealthCheckResult.Healthy("đủ dung lượng")
            : HealthCheckResult.Degraded($"chỉ còn {freeMb} MB");
    })
    .AddCheck("payment-api", () => HealthCheckResult.Unhealthy("không kết nối được (giả lập)"), tags: ["external"]);

var app = builder.Build();

// Bài 2: middleware ghi log mỗi request với scope CorrelationId
app.Use(async (ctx, next) =>
{
    var logger = ctx.RequestServices.GetRequiredService<ILogger<Program>>();
    var correlationId = ctx.Request.Headers["X-Correlation-Id"].FirstOrDefault() ?? "none";
    using (logger.BeginScope("CorrelationId:{CorrelationId}", correlationId))
    {
        await next(ctx);
        Log.RequestFinished(logger, ctx.Request.Method, ctx.Request.Path,
ctx.Response.StatusCode);
    }
});

// Bài 3: tách health check "live" (không kiểm tra phụ thuộc ngoài) và "ready"
app.MapHealthChecks("/health/live", new() { Predicate = c => !c.Tags.Contains("external")
});
app.MapHealthChecks("/health/ready");

await app.StartAsync();
using var http = new HttpClient { BaseAddress = new Uri("http://127.0.0.1:5197") };
http.DefaultRequestHeaders.Add("X-Correlation-Id", "abc-123");
foreach (var path in new[] { "/health/live", "/health/ready" })
{
    var res = await http.GetAsync(path);
    Console.WriteLine($"{path} -> {(int)res.StatusCode} {await
res.Content.ReadAsStringAsync()}");
}
await Task.Delay(200); // cho console logger kịp ghi
await app.StopAsync();

static partial class Log
{
    [LoggerMessage(Level = LogLevel.Information, Message = "{Method} {Path} ->
{StatusCode}")]
    public static partial void RequestFinished(ILogger logger, string method, string path,
int statusCode);
}

```

```

info: Program[856139748] => SpanId:ba8100bedada6937, TraceId:5fcd3b93adf84692a3a3d79ffc575e73,
ParentId:0000000000000000 => ConnectionId:0HNOTEK5QCE1A => RequestPath:/health/live
RequestId:0HNOTEK5QCE1A:00000001 => CorrelationId:abc-123 GET /health/live -> 200
/health/live -> 200 Healthy
fail: Microsoft.Extensions.Diagnostics.HealthChecks.DefaultHealthCheckService[103] =>
SpanId:04a8b4d0e557261b, TraceId:79dbca5fa289906a6e157464bfff16ec6, ParentId:0000000000000000 =>
ConnectionId:0HNOTEK5QCE1A => RequestPath:/health/ready RequestId:0HNOTEK5QCE1A:00000002 =>
CorrelationId:abc-123 => Microsoft.Extensions.Diagnostics.HealthChecks.HealthCheckLogScope Health
check payment-api with status Unhealthy completed after 0.182ms with message 'không kết nối được
(giả lập)'
info: Program[856139748] => SpanId:04a8b4d0e557261b, TraceId:79dbca5fa289906a6e157464bfff16ec6,
ParentId:0000000000000000 => ConnectionId:0HNOTEK5QCE1A => RequestPath:/health/ready

```

```
RequestId:0HN0TEK5QCE1A:00000002 => CorrelationId:abc-123 GET /health/ready -> 503  
/health/ready -> 503 Unhealthy
```

/health/live trả 200 vì bỏ qua check external; /health/ready trả 503 vì "payment-api" Unhealthy. Mỗi dòng log đều mang CorrelationId:abc-123 từ scope.

Nguồn tham khảo (Sources)

- Logging in .NET: <https://github.com/dotnet/docs/blob/main/docs/core/extensions/logging/overview.md>
- Compile-time logging source generation: <https://github.com/dotnet/docs/blob/main/docs/core/extensions/logging/source-generation.md>
- Console log formatting (JSON): <https://github.com/dotnet/docs/blob/main/docs/core/extensions/logging/console-log-formatter.md>
- Logging in ASP.NET Core: <https://github.com/dotnet/AspNetCore.Docs/blob/main/aspnetcore/fundamentals/logging/index.md>
- Health checks in ASP.NET Core: <https://github.com/dotnet/AspNetCore.Docs/blob/main/aspnetcore/host-and-deploy/health-checks.md>
- .NET observability with OpenTelemetry: <https://github.com/dotnet/docs/blob/main/docs/core/diagnostics/observability-with-otel.md>
- OpenTelemetry .NET: <https://github.com/open-telemetry/opentelemetry-dotnet>
- OpenTelemetry.Exporter.Console on NuGet: <https://www.nuget.org/packages/OpenTelemetry.Exporter.Console/1.19.1>

Chương 7 — Docker và dự án cuối

Mục tiêu

- Viết Dockerfile *multi-stage* (nhiều giai đoạn) cho ASP.NET Core, dùng image chính thức từ `mcr.microsoft.com`.
- Build, chạy container, gọi API trong container.
- Chạy an toàn: user không phải root, cấu hình bằng biến môi trường, dữ liệu trong volume.
- Dùng `docker compose` và tổng kết dự án cuối.

Giải thích đơn giản

Docker đóng gói ứng dụng + runtime + thư viện hệ điều hành thành một **image**. Chạy image ra **container** — ở đâu cũng chạy giống nhau (máy bạn, CI, server).

Với Angular, bạn build ra thư mục `dist/` rồi phục vụ bằng nginx. Với .NET:

1. Stage **build**: image SDK (có compiler) → `dotnet publish` ra thư mục `publish/`.
2. Stage **final**: image ASP.NET Core runtime (nhỏ hơn, không có compiler) + copy `publish/`.

Image cuối không chứa source code hay SDK → nhỏ hơn và ít lỗi hỏng hơn.

Ví dụ

Dockerfile

```
# syntax=docker/dockerfile:1
# ----- Stage 1: build & publish (SDK image, ~1 GB, not shipped) -----
FROM mcr.microsoft.com/dotnet/sdk:10.0.401 AS build
WORKDIR /src

# Copy only project files first -> "dotnet restore" layer is cached until a .csproj changes.
COPY global.json Directory.Build.props TaskBoard.slnx ./
COPY src/TaskBoard.Domain/TaskBoard.Domain.csproj src/TaskBoard.Domain/
COPY src/TaskBoard.Application/TaskBoard.Application.csproj src/TaskBoard.Application/
COPY src/TaskBoard.Infrastructure/TaskBoard.Infrastructure.csproj
src/TaskBoard.Infrastructure/
COPY src/TaskBoard.Api/TaskBoard.Api.csproj src/TaskBoard.Api/
# Optional secret "ca_bundle": only needed behind a TLS-inspecting corporate proxy
# (see README). Without the secret this is a normal "dotnet restore".
RUN --mount=type=secret,id=ca_bundle,required=false \
    if [ -f /run/secrets/ca_bundle ]; then export SSL_CERT_FILE=/run/secrets/ca_bundle; fi && \
    dotnet restore src/TaskBoard.Api/TaskBoard.Api.csproj

# Copy the source and publish a Release build.
COPY src/ src/
RUN dotnet publish src/TaskBoard.Api/TaskBoard.Api.csproj -c Release -o /app/publish --no-restore

# ----- Stage 2: runtime (ASP.NET Core runtime only, much smaller) -----
FROM mcr.microsoft.com/dotnet/aspnet:10.0.12 AS final
WORKDIR /app
```

```
# The SQLite file lives in /app/data (mount a volume here to keep data).
RUN mkdir -p /app/data && chown app:app /app/data
ENV ASPNETCORE_HTTP_PORTS=8080 \
    ConnectionStrings__TaskBoard="Data Source=/app/data/taskboard.db"

COPY --from=build /app/publish .

# Run as the non-root "app" user that the official .NET images provide.
USER app
EXPOSE 8080
ENTRYPOINT ["dotnet", "TaskBoard.Api.dll"]
```

```
*/bin/
*/obj/
*/*.db
*/*.db-shm
*/*.db-wal
*/TestResults/
tests/
.git
Dockerfile
```

Điểm chính:

- Tag **cố định** (sdk:10.0.401, aspnet:10.0.12 — có trên mcr.microsoft.com, checked 2026-09-28) để build lặp lại được. Dùng tag 10.0 nếu muốn tự nhận bản vá mới.
- Copy file .csproj trước rồi restore → Docker cache lớp này; sửa code không phải tải lại gói.
- Image ASP.NET Core từ .NET 8 nghe cổng **8080** và có sẵn user **app** không phải root (theo README của image dotnet/aspnet).
- Chuỗi kết nối truyền qua biến môi trường ConnectionStrings__TaskBoard (dấu __ thay cho :).
- Secret ca_bundle là **tùy chọn**: chỉ cần khi build sau proxy công ty có kiểm tra TLS (xem mục "Lỗi và bẫy"). Không có secret thì restore chạy bình thường.

Build, chạy và gọi API — output thật

Script final-project/scripts/docker-smoke.sh build image, chạy container, gọi vài endpoint rồi dọn dẹp:

```
#!/usr/bin/env bash
# Build the Docker image, run it, call a few endpoints, then clean up.
# Behind a TLS-inspecting proxy, set CA_BUNDLE=/path/to/ca-bundle.crt (and HTTPS_PROXY).
set -euo pipefail
cd "$(dirname "$0")/.."
IMAGE=taskboard-api:dev
NAME=taskboard-smoke
PORT=18080

build_args=()
if [[ -n "${CA_BUNDLE:-}" ]]; then
    build_args+=(--network host --secret "id=ca_bundle,src=$CA_BUNDLE")
    [[ -n "${HTTPS_PROXY:-}" ]] && build_args+=(--build-arg "HTTPS_PROXY=$HTTPS_PROXY" --build-arg "HTTPS_PROXY=$HTTPS_PROXY")
fi

echo "\$ docker build -t $IMAGE ."
docker build -q "${build_args[@]}" -t "$IMAGE" . >/dev/null
docker image ls "$IMAGE" --format 'image {{.Repository}}:{{.Tag}} size={{.Size}}'

docker rm -f "$NAME" >/dev/null 2>&1 || true
echo "\$ docker run -d -p $PORT:8080 --name $NAME $IMAGE"
```

```

docker run -d -p "$PORT:8080" --name "$NAME" "$IMAGE" >/dev/null
trap 'docker rm -f "$NAME" >/dev/null 2>&1 || true' EXIT

for i in $(seq 1 30); do
    curl -fsS "http://localhost:$PORT/health" >/dev/null 2>&1 && break
    sleep 1
done

call() { echo "\$ curl $*"; curl -sS -w '\n-> HTTP %{http_code}\n' "$@"; }
call "http://localhost:$PORT/health"
call -X POST "http://localhost:$PORT/api/tasks" -H 'Content-Type: application/json' \
    -d '{"title":"Triển khai bằng Docker","priority":"High","dueDate":"2030-01-31"}'
call -X PATCH "http://localhost:$PORT/api/tasks/1/status" -H 'Content-Type: application/json'
-d '{"status":"Done"}'
call "http://localhost:$PORT/api/tasks?status=Todo"
echo "\$ docker exec $NAME whoami"
docker exec "$NAME" whoami
echo "\$ docker logs $NAME (last 4 lines)"
docker logs "$NAME" 2>&1 | tail -4

```

```

$ docker build -t taskboard-api:dev .
image taskboard-api:dev size=401MB
$ docker run -d -p 18080:8080 --name taskboard-smoke taskboard-api:dev
$ curl http://localhost:18080/health
Healthy
-> HTTP 200
$ curl -X POST http://localhost:18080/api/tasks -H Content-Type: application/json -d {"title":"Triển
khai bằng Docker","priority":"High","dueDate":"2030-01-31"}
{"id":1,"title":"Triển khai bằng
Docker","description":null,"status":"Todo","priority":"High","dueDate":"2030-01-
31","isOverdue":false,"createdAt":"2026-09-28T15:29:28.7572938+00:00","completedAt":null}
-> HTTP 201
$ curl -X PATCH http://localhost:18080/api/tasks/1/status -H Content-Type: application/json -d
{"status":"Done"}
{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.10","title":"Không chuyển được trạng thái từ
Todo sang Done.","status":409,"traceId":"00-75a0b12967265e9f21bfb3137f8c92d1-959a2b6c2ec4bed2-00"}
-> HTTP 409
$ curl http://localhost:18080/api/tasks?status=Todo
{"items":[{"id":1,"title":"Triển khai bằng
Docker","description":null,"status":"Todo","priority":"High","dueDate":"2030-01-
31","isOverdue":false,"createdAt":"2026-09-
28T15:29:28.7572938+00:00","completedAt":null}], "page":1, "pageSize":20, "totalCount":1}
-> HTTP 200
$ docker exec taskboard-smoke whoami
app
$ docker logs taskboard-smoke (last 4 lines)
info: TaskBoard.Application.TaskService[0]
    Created task 1 with priority High
info: TaskBoard.Api.ApiExceptionHandler[0]
    Request PATCH /api/tasks/1/status failed with 409: Không chuyển được trạng thái từ Todo sang
Done.

```

Kết quả cho thấy: container khỏe (Healthy), migration chạy lúc khởi động tạo bảng, POST trả 201, chuyển thẳng Todo → Done bị chặn với 409 (quy tắc Domain), tiến trình chạy bằng user app, và log có cấu trúc.

Đi sâu

Tổng kết dự án cuối TaskBoard

Yêu cầu	Ở đâu	Bảng chứng
Web API CRUD + đổi trạng thái	<code>src/TaskBoard.Api/TaskEndpoints.cs</code>	integration tests, docker smoke
Validation + Problem Details	<code>TaskService.Validate</code> , <code>ApiExceptionHandler</code>	test <code>Post_InvalidBody_Returns400ProblemDetails</code>
EF Core + SQLite + migrations	<code>src/TaskBoard.Infrastructure/</code>	<code>dotnet ef migrations list</code> (Chương 2)
Clean architecture	4 project + <code>ArchitectureTests</code>	Chương 5
Unit + integration tests	<code>tests/</code>	28 test pass (Chương 5)
Health check, OpenAPI, logging	<code>Program.cs</code>	<code>/health</code> , <code>/openapi/v1.json</code> , log trong container
Dockerfile multi-stage, non-root	<code>Dockerfile</code>	output ở trên
Compose + volume	<code>compose.yaml</code>	Bài tập 1

API:

Method	Route	Kết quả
GET	<code>/api/tasks?status=&search=&page=&pageSize=</code>	200 + trang kết quả
GET	<code>/api/tasks/{id}</code>	200 / 404
POST	<code>/api/tasks</code>	201 + Location / 400
PUT	<code>/api/tasks/{id}</code>	200 / 400 / 404 / 409
PATCH	<code>/api/tasks/{id}/status</code>	200 / 404 / 409
DELETE	<code>/api/tasks/{id}</code>	204 / 404
GET	<code>/health</code> , <code>/openapi/v1.json</code>	trạng thái, tài liệu OpenAPI

Kích thước image

Image cuối khoảng 400 MB (số liệu `docker image ls` ở trên). Muốn nhỏ hơn: dùng biến thể `-alpine` hoặc `chiseled` (Ubuntu tối giản, không có shell), hoặc publish Native AOT. Các biến thể này được mô tả trong tài liệu của repo `dotnet/dotnet-docker`. **UNVERIFIED** trong bộ sách: chưa đo kích thước các biến thể này.

Checklist production

- HTTPS ở reverse proxy / load balancer (container chỉ nghe HTTP 8080 bên trong mạng nội bộ).
- Secret (chuỗi kết nối DB thật, API key) qua biến môi trường hoặc secret store, **không** ghi trong image.
- Health check cho orchestrator; log ra stdout (Docker thu thập).
- Database thật (PostgreSQL/SQL Server) thay cho SQLite khi có nhiều instance.

- Chạy migration như một bước deploy riêng khi hệ thống lớn (Chương 2).

Lỗi và bẫy thường gặp

- **Build sau proxy có TLS inspection** (gặp thật khi viết chương này): `dotnet restore` trong container báo NU1301 ... The remote certificate is invalid because of errors in the certificate chain: UntrustedRoot. Container không tin CA của proxy. Cách xử lý không làm bản image: truyền file CA qua BuildKit secret, chỉ dùng trong bước restore: `docker build --secret id=ca_bundle,src=/path/ca-bundle.crt ...` (trong môi trường viết sách còn cần `--network host` và `--build-arg HTTPS_PROXY=...`). Script smoke test làm việc này khi có biến `CA_BUNDLE`.
- **Copy cả thư mục bin/obj vào context** → build chậm, lỗi lạ. Dùng `.dockerignore`.
- **Chạy bằng root** → rủi ro bảo mật. Dùng `USER app`.
- **Lưu SQLite trong container không có volume** → mất dữ liệu khi xóa container.
- **Quên map cổng** (`-p 18080:8080`) → không truy cập được từ máy host.
- **global.json và SDK trong image khác band**: `rollForward: latestFeature` cho phép SDK 10.0.401 trong image build project ghim 10.0.112.

Tóm tắt

- Dockerfile multi-stage: SDK để build, ASP.NET runtime để chạy.
- Tag cố định, cache restore, non-root, cấu hình bằng biến môi trường, dữ liệu trong volume.
- Dự án cuối gom mọi kiến thức Tập 3: Web API, EF Core, clean architecture, test, logging, Docker.

Bài tập (có lời giải)

1. Viết `compose.yaml` chạy API với volume cho thư mục dữ liệu; chứng minh dữ liệu còn sau khi restart container.
2. Vì sao Dockerfile copy các file `.csproj` và chạy `dotnet restore` **trước** khi copy source?
3. Dự án cuối nên đổi gì khi chạy 3 instance API sau load balancer?

▼ Lời giải

Bài 1.

```
# docker compose up -d -> API at http://localhost:8080, data kept in the "taskboard-data"
volume
services:
  api:
    image: taskboard-api:dev
    build: .
    ports:
      - "8080:8080"
    environment:
      ASPNETCORE_ENVIRONMENT: Production
      ConnectionStrings__TaskBoard: "Data Source=/app/data/taskboard.db"
    volumes:
      - taskboard-data:/app/data
    restart: unless-stopped
```

```
volumes:
  taskboard-data:
```

Script kiểm tra (dùng image đã build ở trên):

```
#!/usr/bin/env bash
# Exercise: run the API with docker compose and check that data survives a container
restart.
# Uses the image taskboard-api:dev built by V3Ch07.Docker (so no second build is needed
here).

set -euo pipefail
cd ../../final-project
export COMPOSE_PROJECT_NAME=taskboard-ex
docker compose down -v >/dev/null 2>&1 || true
trap 'docker compose down -v >/dev/null 2>&1 || true' EXIT
echo '$ docker compose up -d --no-build'
docker compose up -d --no-build 2>&1 | grep -v -i "warn" | sed 's/^ *//' | grep -E
"Created|Started|Running" || true
wait_up() { for i in $(seq 1 30); do curl -fsS localhost:8080/health >/dev/null 2>&1 &&
return; sleep 1; done; }
wait_up
echo '$ curl -X POST localhost:8080/api/tasks ...'
curl -sS -o /dev/null -w 'HTTP %{http_code}\n' -X POST localhost:8080/api/tasks -H
'Content-Type: application/json' -d '{"title":"Dữ liệu phải còn sau restart"}'
echo '$ docker compose restart api'
docker compose restart api >/dev/null 2>&1
wait_up
echo '$ curl localhost:8080/api/tasks'
curl -sS localhost:8080/api/tasks | python3 -c 'import json,sys; d=json.load(sys.stdin);
print("totalCount = ", d["totalCount"], " | title = ", d["items"][0]["title"])'
```

```
$ docker compose up -d --no-build
Network taskboard-ex_default Created
Volume taskboard-ex_taskboard-data Created
Container taskboard-ex-api-1 Created
Container taskboard-ex-api-1 Started
$ curl -X POST localhost:8080/api/tasks ...
HTTP 201
$ docker compose restart api
$ curl localhost:8080/api/tasks
totalCount = 1 | title = Dữ liệu phải còn sau restart
```

Bài 2. Docker cache từng lớp (layer). Lớp `restore` chỉ phụ thuộc file `.csproj`; khi bạn chỉ sửa code `.cs`, Docker dùng lại lớp đã `restore` → build nhanh hơn nhiều, không tải lại gói NuGet.

Bài 3. SQLite là file cục bộ — 3 container sẽ có 3 database khác nhau. Cần: database dùng chung (PostgreSQL/SQL Server) bằng cách đổi provider EF Core và chuỗi kết nối; chạy migration một lần trong bước deploy (tắt `Database:MigrateOnStartup`); không giữ trạng thái trong bộ nhớ của từng instance.

Nguồn tham khảo (Sources)

- Containerize a .NET app (nguồn Microsoft Learn): <https://github.com/dotnet/docs/blob/main/docs/core/docker/build-container.md>
- Docker images for ASP.NET Core: <https://github.com/dotnet/AspNetCore.Docs/blob/main/aspnetcore/host-and-deploy/docker/building-net-docker-images.md>

- `dotnet/aspnet` image README (port 8080, non-root): <https://github.com/dotnet/dotnet-docker/blob/main/README.aspnet.md>
- Non-root user sample: <https://github.com/dotnet/dotnet-docker/blob/main/samples/kubernetes/non-root/README.md>
- Ubuntu chiseled images: <https://github.com/dotnet/dotnet-docker/blob/main/documentation/ubuntu-chiseled.md>
- MCR tag list (sdk): <https://mcr.microsoft.com/v2/dotnet/sdk/tags/list>
- Docker build secrets (nguồn docs.docker.com): <https://github.com/docker/docs/blob/main/content/manuals/build/building/secrets.md>
- Dockerfile reference, `RUN --mount=type=secret` (option required): <https://github.com/moby/buildkit/blob/master/frontend/dockerfile/docs/reference.md>

GLOSSARY — Thuật ngữ (Tiếng Anh → Tiếng Việt)

Sắp xếp theo thứ tự chữ cái. Cột "Tập" cho biết thuật ngữ xuất hiện lần đầu ở đâu.

Thuật ngữ	Giải thích tiếng Việt	Tập
abstract class	class trừu tượng, không tạo object trực tiếp, dùng làm lớp cha	1
access modifier	từ khóa phạm vi truy cập: <code>public</code> , <code>private</code> , <code>protected</code> , <code>internal</code>	1
ArrayPool	kho mảng dùng lại, giảm cấp phát bộ nhớ	3
assembly	file <code>.dll</code> / <code>.exe</code> kết quả build một project	1
async / await	cú pháp viết code bất đồng bộ như code tuần tự	1, 2
back-pressure	cơ chế làm bên gửi chậm lại khi bên nhận xử lý không kịp	3
backing field	field ẩn chứa giá trị thật của property	1
BenchmarkDotNet	thư viện đo hiệu năng chuẩn của .NET	3
boxing	đóng gói value type vào object trên heap (tốn bộ nhớ)	2
Channel<T>	hàng đợi bất đồng bộ cho mẫu producer/consumer	3
checked / unchecked	bật/tắt kiểm tra tràn số	1
clean architecture	kiến trúc chia tầng, phụ thuộc chỉ hướng vào trong	3
closure	lambda "nhớ" biến ở phạm vi bên ngoài	2
collection expression	cú pháp <code>[1, 2, 3]</code> tạo collection; <code>..</code> là spread	1
collection	tập hợp nhiều phần tử: <code>List<T></code> , <code>Dictionary</code> , ...	1
constraint (generic)	ràng buộc <code>where T : ...</code> cho tham số kiểu	2
constructor	hàm khởi tạo object	1
container / image (Docker)	image: gói ứng dụng + môi trường; container: image đang chạy	3
DbContext	phiên làm việc với database trong EF Core	3
deadlock	hai luồng chờ nhau mãi mãi	3
decimal	kiểu số thập phân 128 bit, dùng cho tiền	1
deconstruction	tách một giá trị thành nhiều biến: <code>var (a, b) = tuple;</code>	1
deferred execution	thực thi trì hoãn: query LINQ chỉ chạy khi duyệt	2
delegate	kiểu đại diện cho một hàm (con trỏ hàm an toàn kiểu)	2
dependency injection (DI)	tiêm phụ thuộc: object nhận dependency từ bên ngoài thay vì tự tạo	2
Dispose / IDisposable	giải phóng tài nguyên (file, kết nối)	1
DTO (Data Transfer Object)	object chỉ chở dữ liệu request/response	2, 3
EF Core	Entity Framework Core — ORM của .NET	3
endpoint filter	bộ lọc chạy quanh một endpoint Minimal API	3
event	cơ chế thông báo dựa trên delegate	2
exception filter	<code>catch (X) when (...)</code> : chỉ bắt khi điều kiện đúng	1
exception	ngoại lệ, object mô tả lỗi lúc chạy	1

Thuật ngữ	Giải thích tiếng Việt	Tập
extension member	thêm method/property cho kiểu có sẵn mà không sửa kiểu đó (C# 14 mở rộng)	1, 2
field keyword	từ khóa C# 14 truy cập backing field trong property	1
file-based app	ứng dụng chỉ một file <code>.cs</code> , chạy bằng <code>dotnet run app.cs</code> (.NET 10)	1
fixture (test)	dữ liệu/tài nguyên dùng chung giữa các test	2
GC (Garbage Collector)	bộ gom rác, tự giải phóng object không còn dùng	3
Generic Host	"khung" ứng dụng gồm DI, configuration, logging	2
generic	kiểu/hàm có tham số kiểu <code><T></code>	1, 2
health check	endpoint báo ứng dụng còn sống / sẵn sàng	3
heap / stack	vùng nhớ cho object (heap) và cho biến cục bộ, lời gọi hàm (stack)	1
IL (Intermediate Language)	ngôn ngữ trung gian mà C# biên dịch ra	1
immutable	bất biến, không sửa được sau khi tạo	1
init accessor	<code>init</code> : chỉ gán lúc khởi tạo object	1
integration test	test nhiều thành phần cùng chạy thật (HTTP, DB)	3
interface	"hợp đồng" gồm các thành viên mà class phải cài đặt	1
Interlocked	thao tác nguyên tử trên biến số, an toàn đa luồng	3
JIT (Just-In-Time)	biên dịch IL sang mã máy lúc chạy	1
lifetime (DI)	vòng đời service: Singleton, Scoped, Transient	2
LINQ	Language Integrated Query: truy vấn dữ liệu bằng <code>Where</code> , <code>Select</code> ...	1, 2
LTS / STS	Long/Standard Term Support: hỗ trợ dài hạn (3 năm) / ngắn hạn	1
Microsoft.Testing.Platform (MTP)	nền tảng chạy test mới, dùng với xUnit v3 trên .NET 10	2
middleware	thành phần xử lý request/response theo chuỗi	3
migration (EF Core)	file mô tả thay đổi schema database theo thời gian	3
Minimal API	cách viết Web API bằng lambda <code>MapGet/MapPost...</code>	3
namespace	không gian tên, nhóm các kiểu	1
NuGet	kho và trình quản lý gói của .NET	1
null-coalescing	toán tử <code>??</code> và <code>??=</code>	1
null-conditional	toán tử <code>?.</code> và <code>?[]</code>	1
nullable	có thể mang giá trị <code>null</code> : <code>int?</code> , <code>string?</code>	1
observability	khả năng quan sát hệ thống qua logs, traces, metrics	3
OpenTelemetry	chuẩn mở thu thập và xuất logs/traces/metrics	3
ORM	ánh xạ object ↔ bảng database	3
override / virtual	ghi đè method của lớp cha / cho phép ghi đè	1
pattern matching	so khớp mẫu: <code>is</code> , <code>switch</code> với pattern	1, 2
primary constructor	tham số constructor khai báo ngay sau tên class	1
Problem Details	định dạng JSON chuẩn cho lỗi HTTP (RFC 7807, bản mới RFC 9457)	3

Thuật ngữ	Giải thích tiếng Việt	Tập
property	thuộc tính có <code>get / set</code>	1
race condition	kết quả sai do nhiều luồng cùng sửa dữ liệu	3
record	kiểu dữ liệu so sánh theo giá trị, hỗ trợ <code>with</code>	1, 2
reference type	kiểu tham chiếu: gán là copy địa chỉ	1
reified generics	generic giữ thông tin kiểu lúc chạy (khác type erasure của Java)	2
required member	thành viên bắt buộc gán khi khởi tạo	1
runtime	môi trường chạy chương trình (.NET Runtime / CLR)	1
scope (logging)	ngữ cảnh gắn thêm trường cho mọi log bên trong	3
SDK	bộ công cụ phát triển (<code>dotnet CLI, compiler</code>)	1
sealed	không cho kế thừa tiếp	1
<code>Span<T></code>	"cửa sổ" nhìn vào vùng nhớ liên tục, không copy	3
<code>stackalloc</code>	cấp phát bộ nhớ tạm trên stack	3
<code>struct</code>	kiểu giá trị do người dùng định nghĩa	1
structured logging	log có cấu trúc: giữ tên và giá trị từng trường	2, 3
switch expression	<code>x switch { pattern => value, ... }</code> trả về giá trị	1
test double / fake / mock	đối tượng giả thay dependency thật khi test	2
Theory / Fact (xUnit)	test có dữ liệu tham số / test đơn	2
top-level statements	viết code trực tiếp trong <code>Program.cs</code> , không cần <code>Main</code>	1
trace / span	hành trình của một request / một bước trong hành trình đó	3
tuple	nhóm nhiều giá trị: <code>(int Min, int Max)</code>	1
use case	trường hợp sử dụng: một thao tác nghiệp vụ của ứng dụng	3
value type	kiểu giá trị: gán là copy dữ liệu	1
variance (in/out)	quy tắc gán generic giữa kiểu cha/con	2